```
/* ==================== server/index.js ==================== */
/*
 * index.js — 平台服务端（零 npm 依赖，仅使用 Node 内置模块）
 * ================================================================
 * 启动:   node server/index.js [--port 8080] [--host 0.0.0.0]
 * 职责:
 *   · 静态资源托管（public/）
 *   · 账号体系（教师 / 学生，scrypt 加盐哈希）
 *   · 班级-学生批量导入、初始密码管理
 *   · 编译服务（SDCC → .hex，含内置汇编器降级通路）
 *   · 练习记录 / 学情统计
 *   · 提问与答疑（含大模型代理、离线规则助教降级）
 *   · 原理图作品存储
 *
 * 公网/内网部署：默认监听 0.0.0.0，同一局域网内直接可用；
 * 公网访问见 docs/部署指南.md（反向代理 / 内网穿透 / 云托管三种方案）。
 */
import http from 'node:http';
import https from 'node:https';
import fs from 'node:fs';
import path from 'node:path';
import url from 'node:url';
import os from 'node:os';
import zlib from 'node:zlib';
import crypto from 'node:crypto';
import { DB, parseStudentTable } from './db.js';
import { Compiler } from './compile.js';
import { LLM, SYSTEM_TUTOR, SYSTEM_TEACHER, LocalTutor } from './llm.js';
import { PROBLEMS, DEFAULT_SETTINGS } from './seed.js';
const ROOT = path.resolve(import.meta.dirname, '..');
const PUBLIC = path.join(ROOT, 'public');
// 数据目录可用 MCULAB_DATA 覆盖（容器里挂载 volume 时更灵活）
const DATA = process.env.MCULAB_DATA || path.join(ROOT, 'data');
/* ---------------------------------------------------------- 参数 */
const argv = process.argv.slice(2);
const argOf = (k, d) => { const i = argv.indexOf(k); return i >= 0 && argv[i + 1] ? argv[i + 1] : d; };
// PORT 是云平台/容器平台的事实标准（Heroku、Railway、Render、WorkBuddy 发布沙箱等都会注入），
// 优先级：--port > PORT > MCULAB_PORT > 8080
const PORT = Number(argOf('--port', process.env.PORT || process.env.MCULAB_PORT || 8080));
const HOST = argOf('--host', process.env.MCULAB_HOST || '0.0.0.0');
/* ---------------------------------------------------------- TLS
 * 直接对外提供 HTTPS（不依赖 Nginx）。
 *   node server/index.js --tls-cert cert.pem --tls-key key.pem
 * 这么做的意义：Web Serial（真机烧录）只在 HTTPS 或 localhost 下可用；
 * 有了内置 TLS，即使没有反向代理，公网访问也能用真机功能。
 */
const TLS_CERT = argOf('--tls-cert', process.env.MCULAB_TLS_CERT || '');
const TLS_KEY = argOf('--tls-key', process.env.MCULAB_TLS_KEY || '');
const TLS = !!(TLS_CERT && TLS_KEY);
/** 是否处于安全上下文：本机 TLS、或反向代理声明了 https */
```

```
function isSecureReq(req) {
  if (TLS) return true;
  if (req.socket && req.socket.encrypted) return true;
  const xf = String(req.headers['x-forwarded-proto'] || '').split(',')[0].trim().toLowerCase();
  return xf === 'https';
}
/** 会话 Cookie：在 HTTPS 下自动加 Secure；HttpOnly 始终开启 */
/**
 * 会话 Cookie。
 *
 * 默认发**会话级 Cookie**（不设 Max-Age）：浏览器在关闭时丢弃它，但同一浏览器
 * 内的刷新、新开标签页都还在——既保证"刷新后自动进入、实验进度不丢"，
 * 又避免公用电脑上关掉浏览器后仍然保留登录状态。
 *
 * 注意：Chrome/Edge 的"启动时继续上次打开的网页"会连会话 Cookie 一起恢复，
 * 若在公用机上使用，下课后请点「退出」。
 */
function sessionCookie(req, token, maxAge = 0) {
  const secure = isSecureReq(req) ? '; Secure' : '';
  const age = maxAge > 0 ? `; Max-Age=${maxAge}` : '';
  return `mculab_token=${token}; Path=/; HttpOnly; SameSite=Lax${age}${secure}`;
}
const db = new DB(process.env.MCULAB_DB || path.join(DATA, 'mculab.db'));
const compiler = new Compiler();
const llm = new LLM(db);
/* ---------------------------------------------------------------- 初始化 */
function bootstrap() {
  // 设置默认值
  for (const [k, v] of Object.entries(DEFAULT_SETTINGS)) if (db.getSetting(k) == null) db.setSetting(k, v);
  /*
   * 历史数据迁移：早期版本保存平台设置时用 String(v)，布尔会写成 "true"/"false"，
   * 而校验比较的是 '1'，导致"点一次保存就静默关掉名单校验"。
   * 这里把历史值统一成 '1'/'0'，避免继续靠兼容逻辑兜着。
   */
  for (const key of ['require_roster', 'allow_self_register']) {
    const cur = db.getSetting(key, null);
    if (cur == null) continue;
    const s0 = String(cur).trim().toLowerCase();
    if (s0 === 'true' || s0 === 'on' || s0 === 'yes') db.setSetting(key, '1');
    else if (s0 === 'false' || s0 === 'off' || s0 === 'no') db.setSetting(key, '0');
  }
  // 教师账号
  if (!db.sql.prepare("SELECT COUNT(*) n FROM users WHERE role='teacher'").get().n) {
    const pwd = process.env.MCULAB_TEACHER_PWD || 'teacher123';
    db.createUser({ username: 'teacher', password: pwd, role: 'teacher', name: '授课教师' });
    console.log(`\n  ⚠ 已创建默认教师账号：  teacher / ${pwd}\n    请登录后立即在"账号"页修改密码。\n`);
  }
  // 每位教师都必须有一个可用的邀请码（历史库升级后自动补齐）
  for (const t of db.sql.prepare("SELECT id FROM users WHERE role='teacher'").all()) {
    if (!db.activeInviteOf(t.id)) db.createInvite(genInviteCode(), t.id);
```

```
  }
  // 题库：以 seed.js 为准做 UPSERT —— 教师升级平台（改题目/起始代码）后
  // 重启服务即可同步到已有数据库，不必删库重来。
  {
    const upsert = db.sql.prepare(`INSERT INTO problems (id,title,chapter,difficulty,statement,starter,tests
    ,preset,hex,order_no)
      VALUES (?,?,?,?,?,?,?,?,?,?)
      ON CONFLICT(id) DO UPDATE SET
        title=excluded.title, chapter=excluded.chapter, difficulty=excluded.difficulty,
        statement=excluded.statement, starter=excluded.starter, tests=excluded.tests,
        preset=excluded.preset, hex=excluded.hex, order_no=excluded.order_no`);
    let added = 0, updated = 0;
    for (const [i, p] of PROBLEMS.entries()) {
      const existed = db.sql.prepare('SELECT 1 FROM problems WHERE id=?').get(p.id);
      upsert.run(p.id, p.title, p.chapter, p.difficulty, p.statement, p.starter,
        JSON.stringify(p.tests || []), p.preset || '', p.hex || '', i);
      existed ? updated++ : added++;
    }
    console.log(`  题库同步：新增 ${added} 个实验，更新 ${updated} 个。`);
  }
}
bootstrap();
/* ---------------------------------------------------------------- 工具 */
const MIME = {
  '.html': 'text/html; charset=utf-8', '.js': 'text/javascript; charset=utf-8',
  '.mjs': 'text/javascript; charset=utf-8', '.css': 'text/css; charset=utf-8',
  '.json': 'application/json; charset=utf-8', '.svg': 'image/svg+xml',
  '.png': 'image/png', '.jpg': 'image/jpeg', '.ico': 'image/x-icon',
  '.woff2': 'font/woff2', '.txt': 'text/plain; charset=utf-8', '.md': 'text/markdown; charset=utf-8',
  '.hex': 'text/plain; charset=utf-8', '.c': 'text/plain; charset=utf-8',
  '.webmanifest': 'application/manifest+json; charset=utf-8', '.map': 'application/json; charset=utf-8'
};
function send(res, code, body, headers = {}) {
  const buf = Buffer.isBuffer(body) ? body : Buffer.from(typeof body === 'string' ? body : JSON.stringify(bo
    dy), 'utf8');
  res.writeHead(code, { 'Content-Length': buf.length, 'Cache-Control': 'no-store', ...headers });
  res.end(buf);
}
const sendJSON = (res, code, obj) => send(res, code, JSON.stringify(obj), { 'Content-Type': 'application/jso
    n; charset=utf-8' });
/* ------------------------------------------------------------------
 * 传输优化：gzip + ETag + 分级缓存
 * ------------------------------------------------------------------
 * 目标："任意网络"都能用。首屏原本约 810 KB（CodeMirror 就占 400 KB），
 * 在手机流量 / 校园网拥塞时体验很差。这里做三件事：
 *   1. 文本资源 gzip（实测 810 KB → 约 230 KB），压缩结果按 mtime 缓存，避免重复压缩；
 *   2. 静态资源带 ETag，二次访问走 304（弱网下只传几百字节）；
 *   3. 缓存分级：vendor/（不常变）长缓存；业务代码 no-cache + ETag（教师迭代后立即生效）。
 */
const GZIP_CACHE = new Map();       // key: full + mtime + size  → { gz, br }
```

```
const COMPRESSIBLE = /^(text\/|application\/(json|javascript|xml)|image\/svg)/i;
function cachePolicy(rel) {
  // vendor 与预编译 hex 很少变；业务代码必须每次校验（教师随时改）
  if (rel.startsWith('/vendor/')) return 'public, max-age=604800, immutable';
  if (rel.startsWith('/hex/')) return 'public, max-age=3600';
  if (/\.(png|jpg|jpeg|ico|woff2)$/i.test(rel)) return 'public, max-age=604800';
  return 'no-cache';                 // 每次都带 ETag 校验，命中则 304
}
function compress(buf, key) {
  let hit = GZIP_CACHE.get(key);
  if (hit) return hit;
  if (GZIP_CACHE.size > 220) GZIP_CACHE.clear();   // 简单上限，避免无限增长
  hit = {
    gz: buf.length > 1024 ? zlib.gzipSync(buf, { level: 6 }) : null,
    etag: '"' + crypto.createHash('sha1').update(buf).digest('base64url').slice(0, 20) + '"'
  };
  GZIP_CACHE.set(key, hit);
  return hit;
}
/** 发送静态资源：自动处理 gzip、ETag/304、缓存头 */
function sendAsset(req, res, buf, { type, rel }) {
  const key = rel + '|' + buf.length + '|' + (buf.length ? buf[0] + '' + buf[buf.length - 1] : '');
  const { gz, etag } = compress(buf, key);
  const headers = {
    'Content-Type': type,
    'Cache-Control': cachePolicy(rel),
    'ETag': etag,
    'Vary': 'Accept-Encoding'
  };
  // 条件请求：内容未变直接 304，弱网下这是"零字节"响应
  const inm = req.headers['if-none-match'];
  if (inm && inm.split(',').some(t => t.trim() === etag)) {
    res.writeHead(304, headers);
    return res.end();
  }
  const accept = String(req.headers['accept-encoding'] || '');
  if (gz && COMPRESSIBLE.test(type) && /\bgzip\b/.test(accept)) {
    headers['Content-Encoding'] = 'gzip';
    headers['Content-Length'] = gz.length;
    res.writeHead(200, headers);
    return res.end(gz);
  }
  headers['Content-Length'] = buf.length;
  res.writeHead(200, headers);
  res.end(buf);
}
function readBody(req, limit = 8 * 1024 * 1024) {
  return new Promise((resolve, reject) => {
    let n = 0; const chunks = [];
    req.on('data', d => { n += d.length; if (n > limit) { reject(new Error('请求体过大')); req.destroy(); return;
```

```
     } chunks.push(d); });
    req.on('end', () => resolve(Buffer.concat(chunks)));
    req.on('error', reject);
  });
}
async function readJSON(req) {
  const b = await readBody(req);
  if (!b.length) return {};
  try { return JSON.parse(b.toString('utf8')); } catch { throw new Error('请求体不是合法 JSON'); }
}
function cookies(req) {
  const out = {};
  for (const kv of (req.headers.cookie || '').split(';')) {
    const i = kv.indexOf('=');
    if (i > 0) out[kv.slice(0, i).trim()] = decodeURIComponent(kv.slice(i + 1).trim());
  }
  return out;
}
function currentUser(req) {
  const t = cookies(req).mculab_token || req.headers['x-mculab-token'];
  return db.sessionUser(t);
}
function requireRole(req, res, role) {
  const u = currentUser(req);
  if (!u) { sendJSON(res, 401, { error: '未登录或会话已过期' }); return null; }
  if (!u.active) { sendJSON(res, 403, { error: '账号已被停用，请联系教师' }); return null; }
  if (role && u.role !== role) { sendJSON(res, 403, { error: '无权限' }); return null; }
  return u;
}
/* ---------------------------------------------------------------- 路由表 */
const routes = [];
const route = (method, pattern, handler, opts = {}) => {
  const keys = [];
  const re = new RegExp('^' + pattern.replace(/:[A-Za-z_]\w*/g, m => { keys.push(m.slice(1)); return '([^/]+
    )'; }) + '$');
  routes.push({ method, re, keys, handler, opts });
};
/* ============ 认证 ============ */
route('POST', '/api/auth/login', async (req, res) => {
  const { username, password } = await readJSON(req);
  const u = db.getUserByName(normKey(username));
  if (!u || !db.checkPassword(u, String(password || ''))) { db.logEvent(u?.id ?? null, 'login_fail', { usern
    ame }); return sendJSON(res, 401, { error: '账号或密码错误' }); }
  if (!u.active) return sendJSON(res, 403, { error: '账号已停用，请联系教师' });
  const token = db.createSession(u.id);
  db.touchLogin(u.id); db.logEvent(u.id, 'login', {});
  send(res, 200, JSON.stringify({ user: publicUser(u), token }), {
    'Content-Type': 'application/json; charset=utf-8',
    'Set-Cookie': sessionCookie(req, token)
  });
```

```
});
route('POST', '/api/auth/logout', async (req, res) => {
  const t = cookies(req).mculab_token;
  if (t) db.deleteSession(t);
  send(res, 200, JSON.stringify({ ok: true }), { 'Content-Type': 'application/json; charset=utf-8', 'Set-Coo
    kie': 'mculab_token=; Path=/; Max-Age=0' });
});
route('GET', '/api/me', async (req, res) => {
  const u = currentUser(req);
  if (!u) return sendJSON(res, 200, { user: null });
  sendJSON(res, 200, { user: publicUser(u) });
});
route('POST', '/api/auth/change-password', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const { oldPassword, newPassword } = await readJSON(req);
  if (!db.checkPassword(u, String(oldPassword || ''))) return sendJSON(res, 400, { error: '原密码不正确' });
  const p = String(newPassword || '');
  if (p.length < 6) return sendJSON(res, 400, { error: '新密码至少 6 位' });
  db.setPassword(u.id, p, 0);
  sendJSON(res, 200, { ok: true });
});
/* 学生注册：仅当教师已导入其学号+班级时才允许 */
route('POST', '/api/auth/register', async (req, res) => {
  const b = await readJSON(req);
  const uname = normKey(b.username);
  const no = normKey(b.studentNo);
  const cls = normKey(b.className);
  const name = String(b.name || '').trim();
  const pwd = String(b.password || '');
  if (!uname) return sendJSON(res, 400, { error: '请填写登录账号' });
  if (pwd.length < 6) return sendJSON(res, 400, { error: '密码至少 6 位' });
  if (!settingOn('allow_self_register', true)) {
    return sendJSON(res, 403, { error: '教师已关闭自助注册，请使用教师下发的账号密码登录。' });
  }
  const requireRoster = settingOn('require_roster', true);
  const roster = db.getJSON('roster', []);
  const byNoCls = db.sql.prepare("SELECT * FROM users WHERE role='student' AND student_no=? AND class_name=?
    ");
  const byNo = db.sql.prepare("SELECT * FROM users WHERE role='student' AND student_no=?");
  /*
   * 名单校验。这里曾经有个真实漏洞：
   *   if (no && !inRoster && requireRoster) → 403
   * 学号为空时整段判断被短路，于是"什么学号都不填"就能注册进来。
   * 现在改为：启用名单校验时，学号与班级都必须填写且必须命中名单。
   */
  let preAcct = null;
  if (requireRoster) {
    if (!no) return sendJSON(res, 400, { error: '请填写学号（教师已启用名单校验，学号用于核对身份）' });
    if (!cls) return sendJSON(res, 400, { error: '请填写班级（教师已启用名单校验，班级与学号需同时匹配）' });
    const inRoster = roster.some(r => normKey(r.studentNo) === no && normKey(r.className) === cls);
```

```
    preAcct = byNoCls.get(no, cls) || null;
    if (!inRoster && !preAcct) {
      return sendJSON(res, 403, {
        error: '你的学号未在教师导入的名单中。请确认班级与学号是否与教务一致，或联系教师把你加入名单。'
      });
    }
  } else if (no) {
    preAcct = byNo.get(no) || null;
  }
  /* 账号名已被占用 */
  const existing = db.getUserByName(uname);
  if (existing) {
    const sameStudent = existing.role === 'student' && no && normKey(existing.student_no) === no;
    if (!sameStudent) return sendJSON(res, 409, { error: '该账号名已被占用，请换一个，或直接用学号登录' });
    // 同一学号不应挂到两个账号上
    const bound = byNo.get(no);
    if (bound && bound.id !== existing.id) {
      return sendJSON(res, 409, { error: `该学号已绑定账号「${bound.username}」，请联系教师处理。` });
    }
    /*
     * 认领只对"尚未激活"的预建账号开放（必须还处于初始密码状态）。
     * 否则会形成账号劫持：学号本身就是登录名、并不是秘密，
     * 任何人只要知道同学学号，就能来"认领"并把对方密码重置掉。
     * 一旦学生自己设置过密码（must_change_pwd=0），后续只允许登录，
     * 忘记密码需教师重置。
     */
    if (!existing.must_change_pwd) {
      return sendJSON(res, 409, {
        error: '该学号已激活（已设置过密码），请直接登录；忘记密码请联系教师重置。'
      });
    }
    // 学生自己设置的密码不需要再强制修改
    db.setPassword(existing.id, pwd, 0);
    const tk = db.createSession(existing.id);
    db.logEvent(existing.id, 'register_claim', { username: uname, className: cls, studentNo: no });
    return send(res, 200, JSON.stringify({ user: publicUser(db.getUser(existing.id)), token: tk, claimed: tr
    ue }), {
      'Content-Type': 'application/json; charset=utf-8',
      'Set-Cookie': sessionCookie(req, tk)
    });
  }
  /*
   * 学号唯一性：一个学号只能对应一个账号。
   * 否则会出现"同一学号两个账号"（历史上班级留空时就能绕过去），
   * 导致教师看学情时数据被拆散。
   */
  if (no) {
    const used = byNo.get(no);
    if (used) {
      return sendJSON(res, 409, {
```

```
        error: `该学号已开通账号（账号名：${used.username}）。请直接用该账号登录，初始密码见教师通知；忘记密码请联系教师重置。`
      });
    }
  }
  let id;
  if (preAcct) {
    // 名单里有、但账号名还没被占用：接管预建账号，保留其 id（练习记录不丢）
    db.sql.prepare('UPDATE users SET username=?, name=?, class_name=?, student_no=? WHERE id=?')
      .run(uname, name || preAcct.name || '', cls || preAcct.class_name || '', no || preAcct.student_no || '
    ', preAcct.id);
    db.setPassword(preAcct.id, pwd, 1);
    id = preAcct.id;
  } else {
    id = db.createUser({ username: uname, password: pwd, role: 'student', name, className: cls, studentNo: n
    o });
  }
  const token = db.createSession(id);
  db.logEvent(id, 'register', { username: uname, className: cls, studentNo: no });
  send(res, 200, JSON.stringify({ user: publicUser(db.getUser(id)), token }), {
    'Content-Type': 'application/json; charset=utf-8',
    'Set-Cookie': sessionCookie(req, token)
  });
});
/* ---------------------------------------------------------------- 教师邀请码
 * 设计目标：教师可以自助注册，但必须凭已有教师签发的邀请码，
 * 避免学生拿到公网链接后把自己注册成教师。
 *   · 第一个教师（初始化账号）自动获得一个初始邀请码；
 *   · 每位教师都有自己的邀请码，可邀请同事；
 *   · 邀请码可重新生成，旧码立即失效。
 */
function genInviteCode() {
  // 放在函数内部：bootstrap() 在模块顶部执行，模块级 const 此时仍处于 TDZ
  const ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';   // 去掉易混淆的 I/O/0/1
  const pick = () => Array.from({ length: 4 }, () => ALPHABET[Math.floor(Math.random() * ALPHABET.length)]).
    join('');
  return `MCU-${pick()}-${pick()}`;
}
function ensureInviteFor(teacherId) {
  const cur = db.activeInviteOf(teacherId);
  if (cur) return cur.code;
  let code = genInviteCode();
  for (let i = 0; i < 5 && db.findInvite(code); i++) code = genInviteCode();
  return db.createInvite(code, teacherId);
}
/** 是否已有教师（决定教师注册是否需要邀请码） */
route('GET', '/api/auth/teacher-registration', async (req, res) => {
  const n = db.countTeachers();
  sendJSON(res, 200, {
    hasTeacher: n > 0,
    requireInvite: n > 0,
```

```
    teacherCount: n,
    message: n > 0
      ? '教师注册需要邀请码，请向已有教师索取。'
      : '还没有教师账号，第一个注册的教师将自动成为管理员并获得邀请码。'
  });
});
/** 教师注册（需邀请码；首位教师除外） */
route('POST', '/api/auth/register-teacher', async (req, res) => {
  const b = await readJSON(req);
  const uname = normKey(b.username);
  const pwd = String(b.password || '');
  const name = String(b.name || '').trim();
  const code = normKey(b.inviteCode).toUpperCase();
  if (!uname) return sendJSON(res, 400, { error: '请填写登录账号' });
  if (pwd.length < 6) return sendJSON(res, 400, { error: '密码至少 6 位' });
  if (db.getUserByName(uname)) return sendJSON(res, 409, { error: '该账号名已被占用' });
  const first = db.countTeachers() === 0;
  let inv = null;
  if (!first) {
    if (!code) return sendJSON(res, 400, { error: '请填写邀请码（由已有教师提供）' });
    inv = db.findInvite(code);
    if (!inv || !inv.active) return sendJSON(res, 403, { error: '邀请码无效或已停用，请向邀请你的教师重新索取' });
  }
  const id = db.createUser({ username: uname, password: pwd, role: 'teacher', name: name || '教师' });
  if (inv) db.useInvite(inv.code);
  const myCode = ensureInviteFor(id);
  const token = db.createSession(id);
  db.logEvent(id, 'register_teacher', { username: uname, viaInvite: inv ? inv.code : '(first)' });
  send(res, 200, JSON.stringify({
    user: publicUser(db.getUser(id)),
    token,
    inviteCode: myCode,
    isFirstTeacher: first
  }), {
    'Content-Type': 'application/json; charset=utf-8',
    'Set-Cookie': sessionCookie(req, token)
  });
});
/** 我的邀请码 */
route('GET', '/api/teacher/invite', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const code = ensureInviteFor(u.id);
  const inv = db.activeInviteOf(u.id);
  sendJSON(res, 200, { code, uses: inv ? inv.uses : 0, createdAt: inv ? inv.created_at : null });
});
/** 重新生成邀请码（旧码立即失效） */
route('POST', '/api/teacher/invite/regenerate', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  db.deactivateInvites(u.id);
  let code = genInviteCode();
```

```
  for (let i = 0; i < 5 && db.findInvite(code); i++) code = genInviteCode();
  db.createInvite(code, u.id);
  db.logEvent(u.id, 'invite_regenerate', {});
  sendJSON(res, 200, { code, uses: 0 });
});
/** 设置项开关解析：兼容 '1' / 'true' / 'on'（历史数据可能是任意一种） */
function settingOn(key, def = true) {
  const v = db.getSetting(key, null);
  if (v == null) return def;
  const s = String(v).trim().toLowerCase();
  return s === '1' || s === 'true' || s === 'on' || s === 'yes';
}
/* 账号/学号等标识类输入归一化：去空白、全角转半角、去零宽字符。
   中文输入法打出的全角学号/账号肉眼看不出差别，却必然匹配失败。 */
function normKey(v) {
  return String(v == null ? '' : v)
    .replace(/[\uFF01-\uFF5E]/g, (ch) => String.fromCharCode(ch.charCodeAt(0) - 0xFEE0))
    .replace(/\u3000/g, ' ')
    .replace(/[\u200B-\u200D\uFEFF]/g, '')
    .trim();
}
function publicUser(u) {
  return { id: u.id, username: u.username, role: u.role, name: u.name, className: u.class_name, studentNo: u
    .student_no, mustChangePwd: !!u.must_change_pwd };
}
/* ============ 编译 ============ */
route('POST', '/api/compile', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const { code, name, problemId, mcu } = await readJSON(req);
  if (!code || typeof code !== 'string') return sendJSON(res, 400, { error: '缺少源码' });
  if (code.length > 400000) return sendJSON(res, 413, { error: '源码过长（>400KB）' });
  const t0 = Date.now();
  const r = await compiler.compile(code, { name: name || 'main', mcu: mcu || 'mcs51' });
  const ms = Date.now() - t0;
  db.addAttempt({
    userId: u.id, problemId: problemId || '', kind: 'compile', ok: r.ok,
    code: code.slice(0, 20000), hex: r.hex ? r.hex.slice(0, 40000) : '',
    message: r.ok ? `OK via ${r.via} (${ms}ms)` : (r.errors[0]?.msg || '').slice(0, 500)
  });
  sendJSON(res, 200, {
    ok: r.ok, via: r.via, hex: r.hex || null, errors: r.errors || [], warnings: r.warnings || [],
    size: r.size || 0, meta: r.meta || null, ms,
    log: (r.log || []).join('\n').slice(0, 8000)
  });
});
route('GET', '/api/compiler/status', async (req, res) => {
  const p = await compiler.probe();
  // 诊断信息：排查"文件在却执行不了"这类问题（缺动态链接器 / 架构不符 / 丢了可执行位）
  const exe = compiler.sdccExe || '';
  const exists = (() => { try { return fs.existsSync(exe); } catch { return false; } })();
```

```
  let mode = '';
  try { mode = (fs.statSync(exe).mode & 0o777).toString(8); } catch { }
  const loaders = ['/lib64/ld-linux-x86-64.so.2', '/lib/ld-linux-x86-64.so.2', '/lib/ld-musl-x86_64.so.1']
    .filter(x => { try { return fs.existsSync(x); } catch { return false; } });
  let toolchainDirs = 'missing';
  try { toolchainDirs = fs.readdirSync(path.join(ROOT, 'toolchain')).join(','); } catch { }
  sendJSON(res, 200, {
    sdccPresent: compiler.supported, sdccUsable: compiler.available,
    version: p.version || '', error: compiler.probeError || p.error || '',
    assembler: true, toolchainDir: compiler.binDir,
    diag: {
      platform: process.platform, arch: process.arch, release: os.release(),
      exe, exeExists: exists, exeMode: mode, loaders, toolchainDirs
    }
  });
});
/* ============ 题库 ============ */
route('GET', '/api/problems', async (req, res) => {
  const list = db.sql.prepare('SELECT * FROM problems ORDER BY order_no').all();
  const u = currentUser(req);
  const solved = u ? db.sql.prepare("SELECT DISTINCT problem_id FROM attempts WHERE user_id=? AND ok=1").all
    (u.id).map(r => r.problem_id) : [];
  sendJSON(res, 200, { problems: list.map(p => ({ ...p, tests: JSON.parse(p.tests || '[]'), solved: solved.i
    ncludes(p.id) })) });
});
/* ============ 练习记录 ============ */
route('POST', '/api/attempts', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const b = await readJSON(req);
  db.addAttempt({ userId: u.id, problemId: b.problemId, kind: b.kind || 'simulate', ok: b.ok, code: b.code,
    hex: b.hex, message: b.message, cycles: b.cycles });
  sendJSON(res, 200, { ok: true });
});
route('GET', '/api/attempts', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  sendJSON(res, 200, { attempts: db.attemptsOf(u.id, 100) });
});
/* ============ 原理图作品 ============ */
route('GET', '/api/sketches', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  sendJSON(res, 200, { sketches: db.listSketches(u.id) });
});
route('POST', '/api/sketches', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const b = await readJSON(req);
  if (!b.id || !b.json) return sendJSON(res, 400, { error: '缺少 id / json' });
  db.saveSketch({ id: b.id, userId: u.id, problemId: b.problemId, name: b.name || '未命名电路', json: b.json });
  sendJSON(res, 200, { ok: true });
});
route('GET', '/api/sketches/:id', async (req, res, params) => {
```

```
  const u = requireRole(req, res); if (!u) return;
  const s = db.getSketch(params.id);
  if (!s || (s.user_id !== u.id && u.role !== 'teacher')) return sendJSON(res, 404, { error: '未找到' });
  sendJSON(res, 200, { sketch: s });
});
route('DELETE', '/api/sketches/:id', async (req, res, params) => {
  const u = requireRole(req, res); if (!u) return;
  db.deleteSketch(params.id, u.id);
  sendJSON(res, 200, { ok: true });
});
/* ============ 答疑 ============ */
route('POST', '/api/questions', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const b = await readJSON(req);
  if (!b.title) return sendJSON(res, 400, { error: '请填写问题标题' });
  const id = db.addQuestion({ userId: u.id, problemId: b.problemId, title: b.title, body: b.body, code: b.co
    de, context: b.context });
  db.logEvent(u.id, 'question', { id });
  // 自动生成助教回答
  let auto = null;
  const cfg = llm.config;
  if (b.autoAnswer !== false) {
    if (cfg.enabled) {
      try {
        const msgs = [
          { role: 'system', content: SYSTEM_TUTOR + (cfg.systemExtra ? '\n' + cfg.systemExtra : '') },
          { role: 'user', content: buildTutorPrompt(b) }
        ];
        const r = await llm.chat(msgs, { maxTokens: 1600, noThinking: true });
        auto = r.content;
      } catch (e) {
        auto = `（大模型调用失败：${e.message}）\n\n` + LocalTutor.reply({ question: b.title + '\n' + (b.body || ''),
    code: b.code || '', schematic: safeParse(b.context), problem: b.problemId });
      }
    } else {
      auto = LocalTutor.reply({ question: b.title + '\n' + (b.body || ''), code: b.code || '', schematic: sa
    feParse(b.context), problem: b.problemId });
    }
    if (auto) db.addAnswer({ questionId: id, authorId: null, role: 'ai', name: cfg.enabled ? 'AI 助教' : '离线助教
    （规则库）', body: auto });
  }
  sendJSON(res, 200, { id, autoAnswer: auto });
});
route('GET', '/api/questions', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const qs = u.role === 'teacher' ? db.listQuestions({ limit: 200 }) : db.listQuestions({ userId: u.id, limi
    t: 100 });
  sendJSON(res, 200, { questions: qs });
});
route('GET', '/api/questions/:id', async (req, res, params) => {
```

```
  const u = requireRole(req, res); if (!u) return;
  const q = db.question(Number(params.id));
  if (!q) return sendJSON(res, 404, { error: '未找到' });
  if (u.role !== 'teacher' && q.user_id !== u.id) return sendJSON(res, 403, { error: '无权限' });
  sendJSON(res, 200, { question: q, answers: db.answers(q.id) });
});
route('POST', '/api/questions/:id/answer', async (req, res, params) => {
  const u = requireRole(req, res); if (!u) return;
  const q = db.question(Number(params.id));
  if (!q) return sendJSON(res, 404, { error: '未找到' });
  if (u.role !== 'teacher' && q.user_id !== u.id) return sendJSON(res, 403, { error: '无权限' });
  const b = await readJSON(req);
  if (!b.body) return sendJSON(res, 400, { error: '内容为空' });
  /*
   * asAi：云端大模型的回答由前端生成后回传入库。
   * 若按当前登录用户入库，学生提问下会出现"学生自己回答自己"的错误署名，
   * 因此服务端以固定身份记录（不采信前端传来的名字，避免伪造成他人）。
   * 仅限提问者本人使用，教师回答仍按教师身份入库。
   */
  // 归属校验按“是否提问者本人”判断，不限定角色（教师自测时也需要正确署名）
  const asAi = b.asAi === true && q.user_id === u.id;
  const id = db.addAnswer({
    questionId: q.id,
    authorId: asAi ? null : u.id,
    role: asAi ? 'ai' : u.role,
    name: asAi ? '云端 AI 助教' : (u.name || u.username),
    body: b.body
  });
  if (u.role === 'teacher' && b.close) db.setQuestionStatus(q.id, 'answered');
  sendJSON(res, 200, { id });
});
route('POST', '/api/questions/:id/status', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  db.setQuestionStatus(Number(params.id), b.status || 'open');
  sendJSON(res, 200, { ok: true });
});
function buildTutorPrompt(b) {
  const parts = [`【问题】${b.title}`];
  if (b.body) parts.push(`【描述】${b.body}`);
  if (b.problemId) parts.push(`【当前实验】${b.problemId}`);
  if (b.code) parts.push(`【学生代码】\n\`\`\`c\n${String(b.code).slice(0, 6000)}\n\`\`\``);
  const sc = safeParse(b.context);
  if (sc && sc.issues && sc.issues.length) {
    parts.push(`【平台原理图自动诊断结果】\n` + sc.issues.map(i => `- [${i.level}] ${i.msg}`).join('\n'));
  } else if (sc && sc.parts) {
    parts.push(`【原理图概要】元件：${(sc.parts || []).map(p => p.type).join(', ')}；导线 ${sc.wires?.length || 0} 条`);
  }
  parts.push('请分析可能的原因，并用提问的方式引导学生自己找到问题。');
  return parts.join('\n\n');
```

```
}
function safeParse(s) { if (!s) return null; if (typeof s === 'object') return s; try { return JSON.parse(s)
    ; } catch { return null; } }
/* ============ AI 对话（学生直接问代码/硬件） ============ */
route('POST', '/api/ai/chat', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const b = await readJSON(req);
  const cfg = llm.config;
  if (!cfg.enabled) {
    const txt = LocalTutor.reply({ question: b.question || '', code: b.code || '', schematic: safeParse(b.co
    ntext), problem: b.problemId || '' });
    return sendJSON(res, 200, { ok: true, offline: true, content: txt });
  }
  try {
    const msgs = [{ role: 'system', content: SYSTEM_TUTOR + (cfg.systemExtra ? '\n' + cfg.systemExtra : '')
    }];
    for (const m of (b.history || []).slice(-8)) msgs.push({ role: m.role === 'user' ? 'user' : 'assistant',
     content: String(m.content).slice(0, 4000) });
    /*
     * 知识库/联网检索结果由前端一并带上（server 端也做一次兜底检索），
     * 让"自建大模型"这条路径同样能用上教师资料，而不是只有云端路径受益。
     */
    let kbExtra = String(b.kbContext || '');
    if (!kbExtra) {
      try {
        const hits = kbSearch(`${b.question || ''} ${b.body || ''}`, 3);
        if (hits.length) kbExtra = '【教师知识库摘录】\n' + hits.map((h, i) => `(${i + 1}) ${h.text}`).join('\n\n');
      } catch { }
    }
    const userMsg = buildTutorPrompt({ title: b.question || '（无标题）', body: b.body, code: b.code, context: b.
    context, problemId: b.problemId })
      + (kbExtra ? '\n\n' + kbExtra + '\n\n（请优先依据以上资料回答；资料未覆盖时再凭通用经验，并说明这一点。）' : '');
    msgs.push({ role: 'user', content: userMsg });
    const r = await llm.chat(msgs, { noThinking: true });
    sendJSON(res, 200, { ok: true, offline: false, content: r.content, usage: r.usage, model: r.model });
  } catch (e) {
    const txt = `**大模型调用失败**：${e.message}\n\n---\n\n` + LocalTutor.reply({ question: b.question || '', code:
     b.code || '', schematic: safeParse(b.context), problem: b.problemId || '' });
    sendJSON(res, 200, { ok: true, offline: true, degraded: true, content: txt });
  }
});
/* ============ 教师端 ============ */
route('GET', '/api/teacher/students', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  sendJSON(res, 200, { students: db.listStudents(), classes: db.classes() });
});
route('POST', '/api/teacher/students/import', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  const text = b.text || '';
```

```
  if (!text.trim()) return sendJSON(res, 400, { error: '导入内容为空' });
  const { rows } = parseStudentTable(text);
  if (!rows.length) return sendJSON(res, 400, { error: '未能解析出任何学生记录。请确认每行至少包含"学号"。支持的列：学号,姓名,班级,初
  const defaultPwd = b.defaultPassword || '123456';
  const resetExisting = !!b.resetExisting;
  const result = { created: 0, updated: 0, skipped: 0, errors: [], preview: rows.slice(0, 200) };
  const findStu = db.sql.prepare("SELECT * FROM users WHERE role='student' AND student_no=? AND class_name=?
    ");
  for (const r of rows) {
    try {
      if (!r.studentNo) { result.skipped++; continue; }
      const exist = r.className ? findStu.get(r.studentNo, r.className) : db.sql.prepare("SELECT * FROM user
    s WHERE role='student' AND student_no=?").get(r.studentNo);
      const pwd = r.password || defaultPwd;
      if (exist) {
        if (resetExisting) { db.setPassword(exist.id, pwd, 1); result.updated++; }
        else result.skipped++;
      } else {
        db.createUser({ username: r.studentNo, password: pwd, role: 'student', name: r.name, className: r.cl
    assName, studentNo: r.studentNo, mustChange: 1 });
        result.created++;
      }
    } catch (e) { result.errors.push(`${r.studentNo}: ${e.message}`); }
  }
  // 名单快照（供自助注册校验）
  const roster = db.getJSON('roster', []);
  for (const r of rows) if (!roster.some(x => x.studentNo === r.studentNo && x.className === r.className)) r
    oster.push({ studentNo: r.studentNo, className: r.className, name: r.name });
  db.setSetting('roster', roster);
  db.logEvent(u.id, 'import_students', { created: result.created, updated: result.updated, skipped: result.s
    kipped });
  sendJSON(res, 200, { ok: true, result });
});
route('POST', '/api/teacher/students/:id/password', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  const target = db.getUser(Number(params.id));
  if (!target) return sendJSON(res, 404, { error: '未找到该学生' });
  const pwd = String(b.password || '').trim();
  if (pwd.length < 6) return sendJSON(res, 400, { error: '密码至少 6 位' });
  db.setPassword(target.id, pwd, b.mustChange ? 1 : 0);
  db.logEvent(u.id, 'reset_pwd', { target: target.id });
  sendJSON(res, 200, { ok: true });
});
route('POST', '/api/teacher/students/:id/toggle', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const t = db.getUser(Number(params.id));
  if (!t) return sendJSON(res, 404, { error: '未找到' });
  db.sql.prepare('UPDATE users SET active=? WHERE id=?').run(t.active ? 0 : 1, t.id);
  sendJSON(res, 200, { ok: true, active: !t.active });
```

```
});
route('POST', '/api/teacher/students/delete', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  const ids = (b.ids || []).map(Number).filter(Boolean);
  for (const id of ids) db.sql.prepare("DELETE FROM users WHERE id=? AND role='student'").run(id);
  sendJSON(res, 200, { ok: true, deleted: ids.length });
});
route('GET', '/api/teacher/stats', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const stats = { students: db.studentStats(), problems: db.problemStats(), daily: db.dailyActivity(14), cla
    sses: db.classes() };
  sendJSON(res, 200, stats);
});
route('POST', '/api/teacher/analyze', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const stats = { students: db.studentStats(), problems: db.problemStats(), daily: db.dailyActivity(14), cla
    sses: db.classes() };
  const cfg = llm.config;
  if (!cfg.enabled) return sendJSON(res, 200, { ok: true, offline: true, content: LocalTutor.teacherReport(s
    tats) });
  try {
    const r = await llm.chat([
      { role: 'system', content: SYSTEM_TEACHER },
      { role: 'user', content: '以下是《微控制器原理及应用》课程的学生练习数据（JSON）：\n\n```json\n' + JSON.stringify(stats, null
    ).slice(0, 20000) + '\n```\n\n请分析并给出教学建议。' }
    ], { maxTokens: 2600, noThinking: true });
    sendJSON(res, 200, { ok: true, offline: false, content: r.content });
  } catch (e) {
    sendJSON(res, 200, { ok: true, offline: true, degraded: true, content: `**大模型调用失败**：${e.message}\n\n---\
    n\n` + LocalTutor.teacherReport(stats) });
  }
});
route('GET', '/api/teacher/questions', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  sendJSON(res, 200, { questions: db.listQuestions({ limit: 300 }) });
});
/* ============ 实验报告 ============ */
route('GET', '/api/reports', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const q = url.parse(req.url, true).query;
  const r = db.getReport(u.id, q.problemId || '');
  sendJSON(res, 200, { report: r });
});
route('POST', '/api/reports', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const b = await readJSON(req);
  if (!b.content || !String(b.content).trim()) return sendJSON(res, 400, { error: '报告内容为空' });
  const exist = db.getReport(u.id, b.problemId || '');
  if (exist && exist.status === 'reviewed') return sendJSON(res, 409, { error: '该报告已被教师批阅，不能再修改' });
```

```
  const id = db.saveReport({
    userId: u.id, problemId: b.problemId || '', title: b.title || '',
    content: String(b.content).slice(0, 200000),
    status: b.status === 'submitted' ? 'submitted' : 'draft',
    evidence: typeof b.evidence === 'string' ? b.evidence : JSON.stringify(b.evidence || {})
  });
  db.logEvent(u.id, b.status === 'submitted' ? 'report_submit' : 'report_save', { problemId: b.problemId, id
     });
  sendJSON(res, 200, { ok: true, id });
});
route('POST', '/api/reports/generate', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const b = await readJSON(req);
  const p = PROBLEMS.find(x => x.id === b.problemId) || null;
  const ctx = {
    title: b.title || p?.title || '实验报告',
    statement: p?.statement || '',
    code: String(b.code || '').slice(0, 12000),
    evidence: b.evidence || {},
    schematic: b.schematic || null,
    student: { name: u.name || u.username, className: u.class_name || '', studentNo: u.student_no || '' }
  };
  const cfg = llm.config;
  let content = null, offline = true;
  if (cfg.enabled) {
    try {
      const r = await llm.chat([
        {
          role: 'system', content: `你是电子信息类专业实验课教师，负责帮助学生起草《单片机原理及应用》实验报告。
要求：
1. 使用中文，Markdown 格式，包含以下一级标题：一、实验目的；二、实验原理；三、硬件电路设计；四、软件设计；五、实验结果与分析；六、问题与解
2. 依据给定的真实代码、原理图与波形统计撰写，不要编造未给出的数据。
3. "实验结果与分析"要结合代码逻辑与端口/时序现象做定量说明（如延时时间、刷新率、波特率）。
4. "问题与解决"若学生未提供调试记录，请用教学口吻给出该实验最常见的 2-3 个典型问题及排查思路。
5. "心得体会"保留为【请学生补充】占位，不要代写。
6. 代码放在 \`\`\`c 代码块中，只摘录关键段落。`
        },
        { role: 'user', content: buildReportPrompt(ctx) }
      ], { maxTokens: 4096, noThinking: true });
      content = r.content; offline = false;
    } catch (e) {
      content = `> ⚠ 大模型调用失败（${e.message}），以下为离线模板草稿。\n\n` + offlineReport(ctx);
    }
  } else {
    content = offlineReport(ctx);
  }
  const id = db.getReport(u.id, b.problemId || '')?.id || null;
  sendJSON(res, 200, { ok: true, offline, content, id });
});
route('GET', '/api/teacher/reports', async (req, res) => {
```

```
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const q = url.parse(req.url, true).query;
  sendJSON(res, 200, { reports: db.listReports({ problemId: q.problemId || null, status: q.status || null })
     });
});
route('GET', '/api/teacher/reports/:id', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const r = db.report(Number(params.id));
  if (!r) return sendJSON(res, 404, { error: '未找到' });
  sendJSON(res, 200, { report: r });
});
route('POST', '/api/teacher/reports/:id/review', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  db.reviewReport(Number(params.id), { score: b.score, comment: b.comment, status: 'reviewed' });
  sendJSON(res, 200, { ok: true });
});
function buildReportPrompt(c) {
  const ev = c.evidence || {};
  const lines = [
    `【实验名称】${c.title}`,
    `【学生】${c.student.name} ${c.student.className} ${c.student.studentNo}`,
    '',
    '【实验指导书原文】',
    String(c.statement || '（无）').slice(0, 4000)
  ];
  if (c.code) lines.push('', '【学生最终代码】', '```c', c.code, '```');
  if (c.schematic) {
    const s = c.schematic;
    lines.push('', '【硬件电路（由原理图编辑器自动提取）】',
      `主控：${s.mcu || '—'}；元件：${Object.entries(s.parts || {}).map(([k, v]) => `${k}×${v}`).join('、') || '—'}；` +
      `导线 ${s.wires || 0} 条，电气网络 ${s.nets || 0} 个。`);
    if (s.diagnostics?.length) {
      lines.push('自动接线诊断：' + s.diagnostics.map(d => `[${d.level}] ${d.msg}`).join('；'));
    } else {
      lines.push('自动接线诊断：未发现常见错误。');
    }
  }
  lines.push('', '【仿真/调试实测数据】',
    `- 编译：${ev.compileOk ? '成功，' + (ev.hexSize || 0) + ' 字节' : '未成功编译'}`,
    `- 仿真时长：${ev.simCycles ? ev.simCycles + ' 个机器周期（约 ' + (ev.simCycles / 921600).toFixed(3) + ' 秒）' : '未记录'
    }`,
    `- GPIO 跳变次数：${ev.portEvents || 0}`,
    `- 串口记录：${ev.serialLog ? '有（' + ev.serialLog.length + ' 字符）' : '无'}`);
  lines.push('', '请据此撰写实验报告。');
  return lines.join('\n');
}
/** 离线模板：即使没有大模型也能交付一份结构完整、含真实数据的草稿 */
function offlineReport(c) {
```

```
  const ev = c.evidence || {};
  const s = c.schematic || {};
  const code = String(c.code || '');
  const delays = [...code.matchAll(/delay_ms\s*\(\s*(\d+)\s*\)/g)].map(m => Number(m[1]));
  const simSec = ev.simCycles ? (ev.simCycles / 921600) : 0;
  const diagTxt = (s.diagnostics && s.diagnostics.length)
    ? s.diagnostics.map(d => `- [${d.level}] ${d.msg}`).join('\n')
    : '- 平台自动诊断未发现常见接线错误（电源/地/EA/晶振/限流电阻等检查项通过）。';
  return `# ${c.title}
| 项目 | 内容 |
|---|---|
| 姓名 | ${c.student.name} |
| 班级 | ${c.student.className || '—'} |
| 学号 | ${c.student.studentNo || '—'} |
| 日期 | ${new Date().toLocaleDateString('zh-CN')} |
## 一、实验目的
${(c.statement || '').split('\n').filter(l => /目的/.test(l)).join('\n') || '掌握 8051 单片机的 I/O 口控制、定时器与中断的使用方法，
    能够独立完成硬件连接、程序编写与联合调试。'}
## 二、实验原理
本实验基于 STC89C52RC（8051 内核，12T 模式，11.0592 MHz 晶振）。
1 个机器周期 = 12 个振荡周期，因此机器周期频率为 11.0592 MHz / 12 = **921.6 kHz**，即每个机器周期约 **1.085 μs**。
${delays.length ? `程序中使用了 \`delay_ms(${delays[0]})\`，按 12T 模式估算，单次延时约 ${delays[0]} ms（软件延时受编译器优化与循环体
    值需用示波器/逻辑分析仪校准）。` : ''}
## 三、硬件电路设计
- 主控芯片：${s.mcu || 'STC89C52RC'}
- 使用元件：${Object.entries(s.parts || {}).map(([k, v]) => `${k} × ${v}`).join('、') || '—'}
- 导线 ${s.wires || 0} 条，形成 ${s.nets || 0} 个电气网络。
**平台自动接线诊断结果：**
${diagTxt}
## 四、软件设计
\`\`\`c
${code.slice(0, 2600)}${code.length > 2600 ? '\n/* …（完整代码见附件）… */' : ''}
\`\`\`
## 五、实验结果与分析
| 观测项 | 实测/统计值 |
|---|---|
| 编译结果 | ${ev.compileOk ? '成功，' + (ev.hexSize || 0) + ' 字节' : '未成功编译'} |
| 仿真运行时长 | ${simSec ? simSec.toFixed(3) + ' s（' + ev.simCycles + ' 个机器周期）' : '未记录'} |
| GPIO 电平跳变 | ${ev.portEvents || 0} 次 |
| 串口输出 | ${ev.serialLog ? '有数据（' + ev.serialLog.length + ' 字符）' : '无'} |
${ev.portEvents > 100 ? `仿真中记录到 ${ev.portEvents} 次引脚跳变，说明程序确实在持续改变端口状态，符合周期输出的预期。` : '建议在「仿
    」采集波形以获得更充分的时序证据。'}
## 六、问题与解决
> 本节请结合你实际调试中遇到的真实问题填写。以下是该实验的常见问题，供参考排查：
1. **现象**：LED 全亮或全灭，不流动。
   **排查**：确认接法为灌电流（P1 输出 0 才点亮）；若为拉电流接法，初值应取 0x01 而非 0xFE。
2. **现象**：流水太快看不清。
   **排查**：增大延时；注意 \`unsigned int\` 循环在 12T 下约每毫秒数百次空转，需按实测校准。
3. **现象**：仿真能跑但下载到实物不运行。
   **排查**：EA（31 脚）必须接 VCC；晶振与复位电路是否正确；ISP 下载需"先点下载再冷启动"。
```

```
## 七、心得体会
【请学生补充：你在本实验中最主要的收获、遇到的最大困难及解决过程、对硬件与软件协同的理解。】
---
*本报告由平台根据代码、原理图与仿真数据自动生成草稿（离线模板），学生需补充第六、七节后提交。*
`;
}
/* ============ 设置（大模型 / 平台） ============ */
/* ================================================================ 实验任务
 * 教师发布实验要求 + 起止时间；系统记录学生"第一次打开"与"提交"时间，
 * 用两者之差作为该实验用时（超过窗口的部分不计）。
 */
const nowLocal = () => {
  const d = new Date(); const p = n => String(n).padStart(2, '0');
  return `${d.getFullYear()}-${p(d.getMonth() + 1)}-${p(d.getDate())} ${p(d.getHours())}:${p(d.getMinutes())
    }:${p(d.getSeconds())}`;
};
const toDate = (s) => {
  if (!s) return null;
  const t = String(s).replace(' ', 'T');
  const d = new Date(t.length <= 16 ? t + ':00' : t);
  return isNaN(d) ? null : d;
};
/** 用时（分钟，保留 1 位）；超出教师设定窗口的部分截断 */
function minutesBetween(a, b, winStart, winEnd) {
  const s = toDate(a), e = toDate(b);
  if (!s || !e) return null;
  let from = s, to = e;
  const ws = toDate(winStart), we = toDate(winEnd);
  if (ws && from < ws) from = ws;
  if (we && to > we) to = we;
  const m = (to - from) / 60000;
  return m < 0 ? 0 : Math.round(m * 10) / 10;
}
function rowToAssignment(r) {
  return {
    id: r.id, problemId: r.problem_id, title: r.title, requirement: r.requirement || '',
    startAt: r.start_at || '', endAt: r.end_at || '', published: !!r.published,
    createdAt: r.created_at
  };
}
/* ---------------- 学生：可看到的任务 ---------------- */
route('GET', '/api/assignments', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const rows = u.role === 'teacher'
    ? db.sql.prepare('SELECT * FROM assignments ORDER BY id DESC').all()
    : db.sql.prepare('SELECT * FROM assignments WHERE published=1 ORDER BY id DESC').all();
  const my = db.sql.prepare('SELECT assignment_id, kind, at FROM assignment_events WHERE user_id=? ORDER BY
    id ASC').all(u.id);
  const byA = new Map();
  for (const e of my) {
```

```
    if (!byA.has(e.assignment_id)) byA.set(e.assignment_id, {});
    // 打开时间取第一次，提交时间取第一次
    if (!byA.get(e.assignment_id)[e.kind]) byA.get(e.assignment_id)[e.kind] = e.at;
  }
  sendJSON(res, 200, {
    assignments: rows.map(r => {
      const a = rowToAssignment(r);
      const mine = byA.get(r.id) || {};
      const openedAt = mine.open || null, submittedAt = mine.submit || null;
      return {
        ...a, openedAt, submittedAt,
        minutes: openedAt && submittedAt ? minutesBetween(openedAt, submittedAt, a.startAt, a.endAt) : null,
        open: !!(a.startAt && toDate(nowLocal()) >= toDate(a.startAt) && (!a.endAt || toDate(nowLocal()) <=
    toDate(a.endAt)))
      };
    })
  });
});
/* 学生首次打开某实验 → 记录开始时间 */
route('POST', '/api/assignments/:id/open', async (req, res, params) => {
  const u = requireRole(req, res); if (!u) return;
  const a = db.sql.prepare('SELECT * FROM assignments WHERE id=?').get(Number(params.id));
  if (!a) return sendJSON(res, 404, { error: '任务不存在' });
  if (!a.published && u.role !== 'teacher') return sendJSON(res, 403, { error: '任务尚未发布' });
  const exists = db.sql.prepare("SELECT id FROM assignment_events WHERE assignment_id=? AND user_id=? AND ki
    nd='open'").get(a.id, u.id);
  if (!exists) {
    db.sql.prepare("INSERT INTO assignment_events (assignment_id,user_id,kind,at) VALUES (?,?,?,?)").run(a.i
    d, u.id, 'open', nowLocal());
  }
  const at = db.sql.prepare("SELECT at FROM assignment_events WHERE assignment_id=? AND user_id=? AND kind='
    open' ORDER BY id LIMIT 1").get(a.id, u.id);
  sendJSON(res, 200, { openedAt: at ? at.at : null });
});
/* 学生提交 → 记录结束时间 */
route('POST', '/api/assignments/:id/submit', async (req, res, params) => {
  const u = requireRole(req, res); if (!u) return;
  if (u.role === 'teacher') return sendJSON(res, 400, { error: '教师不参与提交' });
  const a = db.sql.prepare('SELECT * FROM assignments WHERE id=?').get(Number(params.id));
  if (!a) return sendJSON(res, 404, { error: '任务不存在' });
  if (!a.published) return sendJSON(res, 403, { error: '任务尚未发布' });
  const open = db.sql.prepare("SELECT at FROM assignment_events WHERE assignment_id=? AND user_id=? AND kind
    ='open' ORDER BY id LIMIT 1").get(a.id, u.id);
  if (!open) {
    db.sql.prepare("INSERT INTO assignment_events (assignment_id,user_id,kind,at) VALUES (?,?,?,?)").run(a.i
    d, u.id, 'open', nowLocal());
  }
  const done = db.sql.prepare("SELECT at FROM assignment_events WHERE assignment_id=? AND user_id=? AND kind
    ='submit' ORDER BY id LIMIT 1").get(a.id, u.id);
  if (!done) {
```

```
    db.sql.prepare("INSERT INTO assignment_events (assignment_id,user_id,kind,at) VALUES (?,?,?,?)").run(a.i
    d, u.id, 'submit', nowLocal());
  }
  const o = db.sql.prepare("SELECT at FROM assignment_events WHERE assignment_id=? AND user_id=? AND kind='o
    pen' ORDER BY id LIMIT 1").get(a.id, u.id);
  const sm = db.sql.prepare("SELECT at FROM assignment_events WHERE assignment_id=? AND user_id=? AND kind='
    submit' ORDER BY id LIMIT 1").get(a.id, u.id);
  sendJSON(res, 200, {
    openedAt: o ? o.at : null,
    submittedAt: sm ? sm.at : null,
    minutes: minutesBetween(o && o.at, sm && sm.at, a.start_at, a.end_at)
  });
});
/* ---------------- 教师：任务管理 ---------------- */
route('GET', '/api/teacher/assignments', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const rows = db.sql.prepare('SELECT * FROM assignments ORDER BY id DESC').all();
  sendJSON(res, 200, {
    assignments: rows.map(rowToAssignment),
    problems: PROBLEMS.map(p => ({ id: p.id, title: p.title }))
  });
});
route('POST', '/api/teacher/assignments', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  const problemId = String(b.problemId || '').trim();
  const title = String(b.title || '').trim();
  if (!problemId || !title) return sendJSON(res, 400, { error: '请选择实验并填写任务名称' });
  const requirement = String(b.requirement || '');
  const startAt = String(b.startAt || '').trim();
  const endAt = String(b.endAt || '').trim();
  if (startAt && endAt && toDate(startAt) > toDate(endAt)) {
    return sendJSON(res, 400, { error: '开始时间不能晚于结束时间' });
  }
  const published = b.published ? 1 : 0;
  if (b.id) {
    db.sql.prepare(`UPDATE assignments SET problem_id=?, title=?, requirement=?, start_at=?, end_at=?, publi
    shed=? WHERE id=?`)
      .run(problemId, title, requirement, startAt, endAt, published, Number(b.id));
    return sendJSON(res, 200, { ok: true, id: Number(b.id) });
  }
  const r = db.sql.prepare(`INSERT INTO assignments (problem_id,title,requirement,start_at,end_at,published,
    created_by)
    VALUES (?,?,?,?,?,?,?)`).run(problemId, title, requirement, startAt, endAt, published, u.id);
  sendJSON(res, 200, { ok: true, id: Number(r.lastInsertRowid) });
});
route('POST', '/api/teacher/assignments/:id/delete', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const id = Number(params.id);
  db.sql.prepare('DELETE FROM assignment_events WHERE assignment_id=?').run(id);
```

```
  db.sql.prepare('DELETE FROM assignments WHERE id=?').run(id);
  sendJSON(res, 200, { ok: true });
});
/* 某任务的学生完成情况（打开/提交/用时） */
route('GET', '/api/teacher/assignments/:id/progress', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const a = db.sql.prepare('SELECT * FROM assignments WHERE id=?').get(Number(params.id));
  if (!a) return sendJSON(res, 404, { error: '任务不存在' });
  const evs = db.sql.prepare('SELECT * FROM assignment_events WHERE assignment_id=? ORDER BY id').all(a.id);
  const byUser = new Map();
  for (const e of evs) {
    if (!byUser.has(e.user_id)) byUser.set(e.user_id, {});
    const m = byUser.get(e.user_id);
    if (!m[e.kind]) m[e.kind] = e.at;
  }
  const students = db.sql.prepare("SELECT id,username,name,class_name,student_no FROM users WHERE role='stud
    ent' ORDER BY class_name, student_no").all();
  sendJSON(res, 200, {
    assignment: rowToAssignment(a),
    rows: students.map(s => {
      const m = byUser.get(s.id) || {};
      return {
        id: s.id, username: s.username, name: s.name, className: s.class_name, studentNo: s.student_no,
        openedAt: m.open || null, submittedAt: m.submit || null,
        minutes: m.open && m.submit ? minutesBetween(m.open, m.submit, a.start_at, a.end_at) : null
      };
    })
  });
});
/* ---------------- 学情明细：某个学生的修改/仿真记录 + 是否在实验窗口内 ----------------
 * 教师最想知道的两件事：
 *   1. 这个学生到底写了什么、改了哪版（每次编译都留了完整代码）；
 *   2. 这些操作是在"教师规定的实验时间内"完成的，还是课外自己练的。
 * 因此把 attempts（代码+结果）与 assignments（时间窗）对齐后一起返回。
 */
route('GET', '/api/teacher/students/:id/activity', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const sid = Number(params.id);
  const st = db.sql.prepare("SELECT id,username,name,class_name,student_no FROM users WHERE id=? AND role='s
    tudent'").get(sid);
  if (!st) return sendJSON(res, 404, { error: '学生不存在' });
  const q = new URL(req.url, 'http://x').searchParams;
  const limit = Math.max(1, Math.min(200, Number(q.get('limit') || 50)));
  const attempts = db.sql.prepare(`SELECT problem_id,kind,ok,code,message,created_at
    FROM attempts WHERE user_id=? ORDER BY id DESC LIMIT ?`).all(sid, limit);
  const asg = db.sql.prepare('SELECT * FROM assignments ORDER BY id DESC').all();
  const evs = db.sql.prepare('SELECT assignment_id,kind,at FROM assignment_events WHERE user_id=?').all(sid)
    ;
  const evOf = new Map();
  for (const e of evs) {
```

```
    if (!evOf.has(e.assignment_id)) evOf.set(e.assignment_id, {});
    const m = evOf.get(e.assignment_id);
    if (!m[e.kind]) m[e.kind] = e.at;
  }
  const ts2 = (s) => toDate(s) ? toDate(s).getTime() : null;
  /** 该时间点落在哪个实验任务的窗口内；返回任务标题，否则 null（= 课外练习） */
  const windowOf = (problemId, at) => {
    const t = ts2(at); if (!t) return null;
    for (const a of asg) {
      if (problemId && a.problem_id !== problemId) continue;
      const s0 = ts2(a.start_at), e0 = ts2(a.end_at);
      if (s0 && t < s0) continue;
      if (e0 && t > e0) continue;
      if (!s0 && !e0) continue;   // 无窗口的任务不用于判定
      return { id: a.id, title: a.title };
    }
    return null;
  };
  sendJSON(res, 200, {
    student: { id: st.id, username: st.username, name: st.name, className: st.class_name, studentNo: st.stud
    ent_no },
    assignments: asg.map(a => {
      const m = evOf.get(a.id) || {};
      return {
        id: a.id, title: a.title, problemId: a.problem_id,
        startAt: a.start_at || '', endAt: a.end_at || '',
        openedAt: m.open || null, submittedAt: m.submit || null,
        minutes: m.open && m.submit ? minutesBetween(m.open, m.submit, a.start_at, a.end_at) : null
      };
    }),
    timeline: attempts.map(a => {
      const win = windowOf(a.problem_id, a.created_at);
      return {
        at: a.created_at, kind: a.kind || 'compile', ok: !!a.ok,
        problemId: a.problem_id || '', message: a.message || '',
        inWindow: !!win, windowTitle: win ? win.title : '',
        code: String(a.code || '').slice(0, 6000)
      };
    })
  });
});
/* 高频编译错误统计 */
route('GET', '/api/teacher/errors', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const rows = db.sql.prepare(`SELECT problem_id, message, COUNT(*) n, COUNT(DISTINCT user_id) users
    FROM attempts WHERE ok=0 AND kind='compile' AND message<>''
    GROUP BY problem_id, message ORDER BY n DESC LIMIT 200`).all();
  const byProblem = new Map();
  for (const r of rows) {
    const pid = r.problem_id || '(未指定)';
```

```
    if (!byProblem.has(pid)) byProblem.set(pid, []);
    byProblem.get(pid).push({ message: r.message, count: r.n, users: r.users });
  }
  sendJSON(res, 200, {
    total: rows.reduce((a, b) => a + b.n, 0),
    groups: [...byProblem.entries()].map(([problemId, items]) => ({ problemId, items: items.slice(0, 10) }))
      .sort((a, b) => b.items[0].count - a.items[0].count)
  });
});
/* ================================================================ 知识库检索
 * 说明（务必如实告知使用者）：
 *   本实现是**关键词相关度检索**（中文按二元组切词 + IDF 加权），不是向量语义检索。
 *   不依赖任何 embedding 服务，因此离线也能用，但同义改写（如"上拉"vs"提拉"）召回较弱。
 *   若以后接入向量模型，只需替换 kbSearch 的打分部分，接口不变。
 */
const KB_STOP = new Set(['的', '了', '和', '是', '在', '我', '有', '就', '不', '人', '都', '一', '上', '也', '很', '到', '说'
    , '要', '去', '会', '着', '没有', '看', '好', '自己', '这', '那', 'the', 'a', 'an', 'of', 'to', 'and', 'is', 'are',
    ', 'in', 'on', 'it']);
/** 中英混排切词：英文按单词，中文按二元组 */
function tokenize(text) {
  const out = [];
  const s = String(text || '').toLowerCase();
  const en = s.match(/[a-z][a-z0-9_]{1,}/g) || [];
  for (const w of en) if (!KB_STOP.has(w) && w.length > 2) out.push(w);
  const zh = s.replace(/[^\u4e00-\u9fa5]+/g, ' ');
  for (const seg of zh.split(/\s+/)) {
    for (let i = 0; i + 1 < seg.length; i++) {
      const bg = seg.slice(i, i + 2);
      if (!KB_STOP.has(bg)) out.push(bg);
    }
  }
  return out;
}
function splitChunks(text, size = 480, overlap = 80) {
  const clean = String(text || '').replace(/\r\n/g, '\n').replace(/[ \t]+\n/g, '\n').trim();
  if (!clean) return [];
  const paras = clean.split(/\n{2,}/).filter(Boolean);
  const chunks = [];
  let buf = '';
  for (const p of paras) {
    if (buf && (buf.length + p.length) > size) {
      chunks.push(buf);
      buf = buf.slice(Math.max(0, buf.length - overlap)) + '\n' + p;
    } else {
      buf = buf ? buf + '\n\n' + p : p;
    }
  }
  if (buf.trim()) chunks.push(buf);
  return chunks.filter(c => c.trim().length >= 20).slice(0, 400);
}
```

```
/** 检索：返回最相关的若干片段 */
function kbSearch(query, limit = 3) {
  const qTokens = tokenize(query);
  if (!qTokens.length) return [];
  const qSet = new Set(qTokens);
  const docs = db.sql.prepare('SELECT id, title, content FROM kb_docs').all();
  const all = [];
  for (const d of docs) {
    for (const c of splitChunks(d.content)) {
      const t = tokenize(c);
      const tf = new Map();
      for (const tk of t) tf.set(tk, (tf.get(tk) || 0) + 1);
      all.push({ docId: d.id, title: d.title, text: c, tf, len: t.length || 1 });
    }
  }
  if (!all.length) return [];
  // IDF：在片段集合上统计，提升"晶振""串口"这类低频词的权重
  const df = new Map();
  for (const ch of all) for (const tk of new Set(ch.tf.keys())) df.set(tk, (df.get(tk) || 0) + 1);
  const N = all.length;
  const idf = (tk) => Math.log(1 + N / (1 + (df.get(tk) || 0)));
  for (const ch of all) {
    let score = 0;
    for (const tk of qSet) {
      const f = ch.tf.get(tk) || 0;
      if (f) score += (1 + Math.log(f)) * idf(tk);
    }
    // 长度归一，避免长片段天然占优
    ch.score = score / Math.sqrt(ch.len);
  }
  return all.filter(c => c.score > 0).sort((a, b) => b.score - a.score).slice(0, limit);
}
/* ---- 知识库管理（教师） ---- */
route('GET', '/api/kb/docs', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const rows = db.sql.prepare('SELECT id,title,filename,chars,created_at,created_by FROM kb_docs ORDER BY id
     DESC').all();
  sendJSON(res, 200, { docs: rows, totalChars: rows.reduce((a, b) => a + (b.chars || 0), 0) });
});
route('POST', '/api/kb/docs', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  const title = String(b.title || '').trim() || String(b.filename || '').trim() || '未命名资料';
  const content = String(b.content || '');
  if (content.trim().length < 20) return sendJSON(res, 400, { error: '内容太短（至少 20 字）' });
  if (content.length > 2 * 1024 * 1024) return sendJSON(res, 400, { error: '单个文件超过 2MB，请拆分后上传' });
  const r = db.sql.prepare('INSERT INTO kb_docs (title,filename,content,chars,created_by) VALUES (?,?,?,?,?)
    ')
    .run(title.slice(0, 120), String(b.filename || '').slice(0, 200), content, content.length, u.id);
  sendJSON(res, 200, { ok: true, id: Number(r.lastInsertRowid), chars: content.length });
```

```
});
route('POST', '/api/kb/docs/:id/delete', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  db.sql.prepare('DELETE FROM kb_docs WHERE id=?').run(Number(params.id));
  sendJSON(res, 200, { ok: true });
});
/* ---- 检索（教师与学生都可用：AI 回答前先查资料） ---- */
route('GET', '/api/kb/search', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const q = String(new URL(req.url, 'http://x').searchParams.get('q') || '');
  if (!q.trim()) return sendJSON(res, 200, { hits: [] });
  const hits = kbSearch(q, 3).map(h => ({ title: h.title, text: h.text, score: Number(h.score.toFixed(3)) })
    );
  sendJSON(res, 200, { hits });
});
/* ================================================================ 联网搜索
 * 采用可插拔设计：教师在后台上填服务商与密钥。未配置时接口明确返回"未启用"，
 * 不假装能搜。当前内置 Tavily（https://tavily.com）这一种，接口简单、按次计费。
 */
function getWebSearchCfg() {
  const get = (k) => (db.getSetting ? db.getSetting(k) : null);
  return {
    enabled: String(get('websearch_enabled') || '0') === '1',
    provider: get('websearch_provider') || 'tavily',
    apiKey: get('websearch_key') || '',
    maxResults: Number(get('websearch_max') || 5)
  };
}
route('GET', '/api/websearch/config', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const c = getWebSearchCfg();
  sendJSON(res, 200, { enabled: c.enabled, provider: c.provider, hasKey: !!c.apiKey, maxResults: c.maxResult
    s });
});
route('POST', '/api/websearch/config', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  db.setSetting('websearch_enabled', b.enabled ? '1' : '0');
  db.setSetting('websearch_provider', String(b.provider || 'tavily').slice(0, 32));
  if (typeof b.apiKey === 'string' && b.apiKey.trim()) db.setSetting('websearch_key', b.apiKey.trim());
  if (b.maxResults) db.setSetting('websearch_max', String(Math.max(1, Math.min(10, Number(b.maxResults) || 5
    ))));
  sendJSON(res, 200, { ok: true });
});
async function tavilySearch(q, maxResults) {
  const c = getWebSearchCfg();
  if (!c.apiKey) throw new Error('未配置搜索服务密钥');
  const r = await fetch('https://api.tavily.com/search', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
```

```
    body: JSON.stringify({ api_key: c.apiKey, query: q, max_results: maxResults, search_depth: 'basic' })
  });
  if (!r.ok) throw new Error('搜索服务返回 ' + r.status);
  const j = await r.json();
  return (j.results || []).map(x => ({ title: x.title, url: x.url, snippet: String(x.content || '').slice(0,
     600) }));
}
route('GET', '/api/websearch/search', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const c = getWebSearchCfg();
  if (!c.enabled) return sendJSON(res, 200, { enabled: false, hits: [], reason: '教师尚未启用联网搜索' });
  if (!c.apiKey) return sendJSON(res, 200, { enabled: false, hits: [], reason: '联网搜索已启用但缺少服务密钥' });
  const q = String(new URL(req.url, 'http://x').searchParams.get('q') || '').trim();
  if (!q) return sendJSON(res, 200, { enabled: true, hits: [] });
  try {
    const hits = await tavilySearch(q, c.maxResults);
    sendJSON(res, 200, { enabled: true, hits });
  } catch (e) {
    sendJSON(res, 200, { enabled: true, hits: [], reason: e.message });
  }
});
/* ================================================================ 开放性命题
 * 教师自由出题（不绑定内置 15 个实验），学生提交"原理图 + 代码 + 报告"三件套，
 * 教师给一个总分并写评语（开放性命题不拆子项，因为评分标准随题目而变）。
 */
function rowToOpenProblem(r, sub) {
  return {
    id: r.id, title: r.title, statement: r.statement || '',
    startAt: r.start_at || '', endAt: r.end_at || '', published: !!r.published,
    createdAt: r.created_at,
    submission: sub ? {
      id: sub.id, submittedAt: sub.submitted_at,
      score: sub.score, comment: sub.comment || '', gradedAt: sub.graded_at,
      hasSchematic: !!sub.schematic, hasCode: !!sub.code, hasReport: !!sub.report
    } : null
  };
}
/* 学生/教师：可见的开放题（学生只看已发布） */
route('GET', '/api/open-problems', async (req, res) => {
  const u = requireRole(req, res); if (!u) return;
  const rows = u.role === 'teacher'
    ? db.sql.prepare('SELECT * FROM open_problems ORDER BY id DESC').all()
    : db.sql.prepare('SELECT * FROM open_problems WHERE published=1 ORDER BY id DESC').all();
  const subs = u.role === 'student'
    ? db.sql.prepare('SELECT * FROM open_submissions WHERE user_id=?').all(u.id)
    : [];
  const byP = new Map(subs.map(x => [x.problem_id, x]));
  sendJSON(res, 200, { problems: rows.map(r => rowToOpenProblem(r, byP.get(r.id))) });
});
/* 教师：新建/编辑/删除 */
```

```
route('POST', '/api/teacher/open-problems', async (req, res) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const b = await readJSON(req);
  const title = String(b.title || '').trim();
  if (!title) return sendJSON(res, 400, { error: '请填写命题名称' });
  const statement = String(b.statement || '');
  const startAt = String(b.startAt || '').trim();
  const endAt = String(b.endAt || '').trim();
  if (startAt && endAt && toDate(startAt) > toDate(endAt)) return sendJSON(res, 400, { error: '开始时间不能晚于结束时间'
     });
  const published = b.published ? 1 : 0;
  if (b.id) {
    db.sql.prepare('UPDATE open_problems SET title=?, statement=?, start_at=?, end_at=?, published=? WHERE i
    d=?')
      .run(title, statement, startAt, endAt, published, Number(b.id));
    return sendJSON(res, 200, { ok: true, id: Number(b.id) });
  }
  const r = db.sql.prepare('INSERT INTO open_problems (title,statement,start_at,end_at,published,created_by)
     VALUES (?,?,?,?,?,?)')
    .run(title, statement, startAt, endAt, published, u.id);
  sendJSON(res, 200, { ok: true, id: Number(r.lastInsertRowid) });
});
route('POST', '/api/teacher/open-problems/:id/delete', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const id = Number(params.id);
  db.sql.prepare('DELETE FROM open_submissions WHERE problem_id=?').run(id);
  db.sql.prepare('DELETE FROM open_problems WHERE id=?').run(id);
  sendJSON(res, 200, { ok: true });
});
/* 学生：查看自己的提交（含完整内容，用于回填表单） */
route('GET', '/api/open-problems/:id/mine', async (req, res, params) => {
  const u = requireRole(req, res); if (!u) return;
  const p = db.sql.prepare('SELECT * FROM open_problems WHERE id=?').get(Number(params.id));
  if (!p) return sendJSON(res, 404, { error: '命题不存在' });
  if (!p.published && u.role !== 'teacher') return sendJSON(res, 403, { error: '命题尚未发布' });
  const sub = u.role === 'student'
    ? db.sql.prepare('SELECT * FROM open_submissions WHERE problem_id=? AND user_id=?').get(p.id, u.id)
    : null;
  sendJSON(res, 200, {
    problem: rowToOpenProblem(p, sub),
    submission: sub ? { schematic: sub.schematic, code: sub.code, report: sub.report, submittedAt: sub.submi
    tted_at, score: sub.score, comment: sub.comment } : null
  });
});
/* 学生：提交/更新三件套（可重复提交，覆盖前一次，但有提交时间记录） */
route('POST', '/api/open-problems/:id/submit', async (req, res, params) => {
  const u = requireRole(req, res); if (!u) return;
  if (u.role !== 'student') return sendJSON(res, 400, { error: '教师不参与提交' });
  const p = db.sql.prepare('SELECT * FROM open_problems WHERE id=?').get(Number(params.id));
  if (!p) return sendJSON(res, 404, { error: '命题不存在' });
```

```
  if (!p.published) return sendJSON(res, 403, { error: '命题尚未发布' });
  const b = await readJSON(req);
  const schematic = typeof b.schematic === 'string' ? b.schematic : JSON.stringify(b.schematic || '');
  const code = String(b.code || '');
  const report = String(b.report || '');
  if (!code.trim() && !report.trim() && !schematic.trim()) {
    return sendJSON(res, 400, { error: '代码、报告、原理图至少要提交一项' });
  }
  const at = nowLocal();
  const exist = db.sql.prepare('SELECT id FROM open_submissions WHERE problem_id=? AND user_id=?').get(p.id,
     u.id);
  if (exist) {
    db.sql.prepare(`UPDATE open_submissions SET schematic=?, code=?, report=?, submitted_at=? WHERE id=?`)
      .run(schematic, code, report, at, exist.id);
  } else {
    db.sql.prepare(`INSERT INTO open_submissions (problem_id,user_id,schematic,code,report,submitted_at)
      VALUES (?,?,?,?,?,?)`).run(p.id, u.id, schematic, code, report, at);
  }
  sendJSON(res, 200, { ok: true, submittedAt: at });
});
/* 教师：某命题的全部提交 */
route('GET', '/api/teacher/open-problems/:id/submissions', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const p = db.sql.prepare('SELECT * FROM open_problems WHERE id=?').get(Number(params.id));
  if (!p) return sendJSON(res, 404, { error: '命题不存在' });
  const rows = db.sql.prepare(`SELECT s.*, u.username, u.name, u.class_name, u.student_no
    FROM open_submissions s JOIN users u ON u.id = s.user_id
    WHERE s.problem_id=? ORDER BY s.submitted_at DESC`).all(p.id);
  const total = db.sql.prepare("SELECT COUNT(*) n FROM users WHERE role='student'").get().n;
  sendJSON(res, 200, {
    problem: rowToOpenProblem(p, null),
    totalStudents: total,
    rows: rows.map(r => ({
      id: r.id, userId: r.user_id, username: r.username, name: r.name,
      className: r.class_name, studentNo: r.student_no,
      submittedAt: r.submitted_at, score: r.score, comment: r.comment || '',
      hasSchematic: !!r.schematic, hasCode: !!r.code, hasReport: !!r.report
    }))
  });
});
/* 教师：查看某份提交的完整内容 */
route('GET', '/api/teacher/open-submissions/:id', async (req, res, params) => {
  const u = requireRole(req, res, 'teacher'); if (!u) return;
  const r = db.sql.prepare(`SELECT s.*, u.username, u.name, p.title
    FROM open_submissions s JOIN users u ON u.id=s.user_id JOIN open_problems p ON p.id=s.problem_id
    WHERE s.id=?`).get(Number(params.id));
  if (!r) return sendJSON(res, 404, { error: '提交不存在' });
  sendJSON(res, 200, {
    id: r.id, title: r.title, name: r.name, username: r.username,
    schematic: r.schematic, code: r.code, report: r.report,
```

```
  renderPager(rows.length, totalPages, start, view.length);
}
function updatePagerSel() { renderPager(
  students.filter(x => !classFilter || x.class_name === classFilter).length,
  Math.max(1, Math.ceil(students.filter(x => !classFilter || x.class_name === classFilter).length / PAGE_SIZ
    E)),
  (page - 1) * PAGE_SIZE,
  Math.min(PAGE_SIZE, students.filter(x => !classFilter || x.class_name === classFilter).length - (page - 1)
     * PAGE_SIZE)
); }
function renderPager(total, totalPages, start, shown) {
  const box = $('#studentPager');
  if (!box) return;
  if (!total) { box.innerHTML = ''; return; }
  box.innerHTML = `
    <span class="small muted">
      共 ${total} 人${classFilter ? '（' + esc(classFilter) + '）' : ''}，第 ${page} / ${totalPages} 页
      ${shown ? `，本页显示第 ${start + 1}–${start + shown} 人` : ''}
      ${selected.size ? `  <b>已选 ${selected.size} 人</b>` : ''}
    </span>
    <button class="btn sm" data-page="prev" ${page <= 1 ? 'disabled' : ''}>上一页</button>
    <button class="btn sm" data-page="next" ${page >= totalPages ? 'disabled' : ''}>下一页</button>`;
}
function fmtShort(t) {
  if (!t) return '—';
  const d = new Date(String(t).replace(' ', 'T'));
  return isNaN(d) ? String(t) : d.toLocaleString('zh-CN', { month: '2-digit', day: '2-digit', hour: '2-digit
    ', minute: '2-digit' });
}
/* ================================================================ 操作 */
async function onTableClick(e) {
  const btn = e.target.closest('button[data-act]');
  if (!btn) return;
  const id = Number(btn.dataset.id);
  const stu = students.find(s => s.id === id);
  if (!stu) return;
  if (btn.dataset.act === 'detail') { showStudentActivity(Number(btn.dataset.id)); return; }
  if (btn.dataset.act === 'pwd') {
    const r = await promptBox('重置学生密码', [
      { name: 'pwd', label: `${stu.name || stu.username} 的新密码`, value: '123456' }
    ]);
    if (!r || !r.pwd) return;
    try {
      await api(`/api/teacher/students/${id}/password`, { method: 'POST', body: { password: r.pwd, mustChang
    e: true } });
      toast('密码已重置（该生下次登录需修改）', 'ok');
    } catch (err) { toast(err.message, 'err'); }
  }
  if (btn.dataset.act === 'toggle') {
    try {
```

```
      await api(`/api/teacher/students/${id}/toggle`, { method: 'POST', body: { active: !stu.active } });
      await loadStudents();
      toast(stu.active ? '已停用' : '已启用', 'ok');
    } catch (err) { toast(err.message, 'err'); }
  }
}
async function doImport() {
  const text = $('#importText').value;
  if (!text.trim()) return toast('请先粘贴名单', 'warn');
  const btn = $('#btnImport');
  btn.classList.add('busy');
  try {
    const r = await api('/api/teacher/students/import', {
      method: 'POST',
      body: {
        text,
        defaultPassword: $('#importDefaultPwd').value || '123456',
        resetExisting: $('#chkResetExisting').checked
      }
    });
    const x = r.result;
    $('#importResult').innerHTML = `
      <div class="diag-item ok"><b>导入完成</b>
        新建 ${x.created} 人 · 更新 ${x.updated} 人 · 跳过 ${x.skipped} 人
        ${x.errors?.length ? `<div class="err small">${x.errors.length} 条错误：${esc(x.errors.slice(0, 3).join(
    '; '))}</div>` : ''}
      </div>`;
    toast(`导入完成：新增 ${x.created} 人`, 'ok');
    await loadStudents(); await loadStats();
  } catch (e) { toast(e.message, 'err'); }
  finally { btn.classList.remove('busy'); }
}
async function analyze() {
  const btn = $('#btnAnalyzeClass');
  btn.classList.add('busy');
  $('#analysisOut').innerHTML = '<span class="muted">分析中…（离线模式为即时生成）</span>';
  try {
    const r = await api('/api/teacher/analyze', { method: 'POST', body: {} });
    $('#analysisOut').innerHTML = md(r.content || '');
    if (r.offline) toast('当前为离线规则分析（启用大模型可获得更深入的建议）', 'warn');
  } catch (e) { $('#analysisOut').innerHTML = `<div class="diag-item err">${esc(e.message)}</div>`; }
  finally { btn.classList.remove('busy'); }
}
function exportCsv() {
  const stMap = new Map((stats?.students || []).map(s => [s.id, s]));
  const rows = [['学号', '姓名', '班级', '练习次数', '通过次数', '完成实验数', '提问数', '最近活动', '状态']];
  for (const s of students.filter(s => !classFilter || s.class_name === classFilter)) {
    const x = stMap.get(s.id) || {};
    rows.push([s.student_no || s.username, s.name || '', s.class_name || '', x.attempts || 0, x.ok_count ||
    0, x.solved || 0, x.questions || 0, x.last_attempt || s.last_login || '', s.active ? '正常' : '停用']);
```

```
  }
  const csv = '\uFEFF' + rows.map(r => r.map(v => `"${String(v).replace(/"/g, '""')}"`).join(',')).join('\r\
    n');
  const blob = new Blob([csv], { type: 'text/csv;charset=utf-8' });
  const a = el('a', { href: URL.createObjectURL(blob), download: `学情_${classFilter || '全部'}_${new Date().toI
    SOString().slice(0, 10)}.csv` });
  document.body.append(a); a.click(); a.remove();
  setTimeout(() => URL.revokeObjectURL(a.href), 3000);
  toast('已导出 CSV（Excel 可直接打开）', 'ok');
}
async function answerQuestion(qid) {
  try {
    const r = await api('/api/questions/' + qid);
    const q = r.question, ans = r.answers || [];
    const wrap = document.createElement('div');
    wrap.innerHTML = `
      <div class="small muted">${esc(q.uname || '')} · ${esc(q.created_at || '')} · 状态 ${esc(q.status || '')
    }</div>
      <h3 style="margin:8px 0">${esc(q.title)}</h3>
      <div class="md">${md(q.body || '')}</div>
      ${q.code ? `<pre class="code-preview">${esc(q.code.slice(0, 3000))}</pre>` : ''}
      <h3 style="margin:12px 0 6px">已有回复</h3>
      ${ans.map(a => `<div class="ev-item"><b>${esc(a.author_name || a.author_role || '')}</b><div class="md
     small">${md(a.body || '')}</div></div>`).join('') || '<div class="muted small">暂无</div>'}
      <label style="margin-top:12px">我的答复<textarea id="tAns" class="import-box"></textarea></label>
      <div class="row" style="justify-content:flex-end;gap:8px;margin-top:8px">
        <button class="btn" id="tAnsClose">保存并标记已答复</button>
        <button class="btn primary" id="tAnsSend">发送答复</button>
      </div>`;
    modal('学生提问', wrap);
    const send = async (close) => {
      const body = $('#tAns').value.trim();
      if (!body) return toast('答复内容为空', 'warn');
      try {
        await api(`/api/questions/${qid}/answer`, { method: 'POST', body: { body, close } });
        toast('已发送', 'ok');
        closeModal(); loadTeacherQA();
      } catch (e) { toast(e.message, 'err'); }
    };
    $('#tAnsSend').addEventListener('click', () => send(false));
    $('#tAnsClose').addEventListener('click', () => send(true));
  } catch (e) { toast(e.message, 'err'); }
}
/* ================================================================ 设置 */
async function saveLLM() {
  const btn = $('#btnSaveLLM');
  btn.classList.add('busy');
  try {
    await api('/api/settings/llm', {
      method: 'POST',
```

```
      body: {
        enabled: $('#llmEnabled').checked,
        baseUrl: $('#llmBaseUrl').value.trim(),
        model: $('#llmModel').value.trim(),
        apiKey: $('#llmApiKey').value.trim(),
        systemExtra: $('#llmSystemExtra').value.trim()
      }
    });
    toast('大模型设置已保存（密钥仅存服务端）', 'ok');
    await loadSettings();
  } catch (e) { toast(e.message, 'err'); }
  finally { btn.classList.remove('busy'); }
}
async function testLLM() {
  const box = $('#llmTestOut');
  box.innerHTML = '<span class="muted">测试中…</span>';
  try {
    const r = await api('/api/settings/llm/test', { method: 'POST', body: {} });
    box.innerHTML = `<div class="diag-item ${r.ok ? 'ok' : 'err'}"><b>${r.ok ? '连接成功' : '连接失败'}</b>${esc(r.m
    essage || r.reply || '')}</div>`;
  } catch (e) { box.innerHTML = `<div class="diag-item err">${esc(e.message)}</div>`; }
}
async function savePlatform() {
  try {
    await api('/api/settings/platform', {
      method: 'POST',
      body: {
        requireRoster: $('#chkRequireRoster').checked,
        allowSelfRegister: $('#chkAllowRegister').checked,
        siteName: $('#siteNameInput').value.trim(),
        announce: $('#announceInput').value.trim()
      }
    });
    toast('平台设置已保存', 'ok');
    const info = await api('/api/info');
    document.title = info.siteName;
    $('#siteName').textContent = info.siteName;
    $('#siteAnnounce').textContent = info.announce || '';
  } catch (e) { toast(e.message, 'err'); }
}
/* ---------------------------------------------------------------- 教师邀请码 */
async function loadInvite() {
  try {
    const r = await api('/api/teacher/invite');
    const elCode = $('#inviteCodeText');
    if (elCode) elCode.textContent = r.code;
    const elUses = $('#inviteUses');
    if (elUses) elUses.textContent = r.uses ? `已被使用 ${r.uses} 次` : '尚未被使用';
  } catch (e) {
    const elCode = $('#inviteCodeText');
```

```
    if (elCode) elCode.textContent = '加载失败';
  }
}
async function copyInvite() {
  const code = ($('#inviteCodeText')?.textContent || '').trim();
  if (!code || code === '—') return toast('邀请码尚未加载', 'warn');
  try {
    await navigator.clipboard.writeText(code);
    toast('邀请码已复制：' + code, 'ok');
  } catch {
    // 非安全上下文（http）下 clipboard 不可用，退回选中提示
    toast('请手动复制：' + code, 'warn', 5000);
  }
}
async function regenInvite() {
  const ok = await confirmBox('重新生成邀请码', '旧邀请码将立即失效，已注册的教师不受影响。确定重新生成吗？');
  if (!ok) return;
  try {
    const r = await api('/api/teacher/invite/regenerate', { method: 'POST' });
    const elCode = $('#inviteCodeText');
    if (elCode) elCode.textContent = r.code;
    const elUses = $('#inviteUses');
    if (elUses) elUses.textContent = '尚未被使用';
    toast('已生成新邀请码：' + r.code, 'ok', 6000);
  } catch (e) { toast('生成失败：' + e.message, 'err'); }
}
/* 厂商预设：把官方接口地址与常用模型名填好，教师只需粘贴密钥 */
const LLM_PRESETS = {
  // 注意：deepseek-chat / deepseek-reasoner 已于 2026-07-24 停用，现行模型名为 v4 系列
  deepseek: { baseUrl: 'https://api.deepseek.com/v1', model: 'deepseek-v4-flash', hint: '在 platform.deepseek
    .com 创建 API Key' },
  glm: { baseUrl: 'https://open.bigmodel.cn/api/paas/v4', model: 'glm-4-flash', hint: '在 open.bigmodel.cn 创建
     API Key（glm-4-flash 有免费额度）' },
  openai: { baseUrl: 'https://api.openai.com/v1', model: 'gpt-4o-mini', hint: '需海外网络环境' },
  ollama: { baseUrl: 'http://127.0.0.1:11434/v1', model: 'qwen2.5:7b', hint: '本机运行 Ollama，密钥留空即可' }
};
function applyLlmPreset(name) {
  const p = LLM_PRESETS[name];
  if (!p) return;
  const bu = $('#llmBaseUrl'), md = $('#llmModel'), out = $('#llmTestOut');
  if (bu) bu.value = p.baseUrl;
  if (md) md.value = p.model;
  if (out) out.textContent = '已填入 ' + p.baseUrl + ' 与 ' + p.model + '。' + p.hint + '，粘贴密钥后点「保存设置」再「测试连接」。
}
/* ---------------------------------------------------------------- 高频错误统计 */
let errorsLoaded = false;
async function loadErrors(notify) {
  const box = $('#errorStats');
  if (!box) return;
  try {
```

```
    const r = await api('/api/teacher/errors');
    errorsLoaded = true;
    if (!r.groups || !r.groups.length) {
      box.innerHTML = '<div class="empty">还没有编译失败记录。学生编译出错后，这里会自动聚合。</div>';
      return;
    }
    const groups = r.groups.map(g => {
      const rows = g.items.map(it => `
        <tr>
          <td class="small" style="max-width:420px;word-break:break-all">${esc(it.message)}</td>
          <td><b>${it.count}</b></td>
          <td>${it.users}</td>
        </tr>`).join('');
      return `
        <div class="err-group">
          <div class="err-group-title">${esc(g.problemId)}<span class="muted small">（共 ${g.items.reduce((a,
    b) => a + b.count, 0)} 次）</span></div>
          <table class="data-table"><thead><tr><th>错误信息</th><th>次数</th><th>涉及人数</th></tr></thead><tbody>${ro
    ws}</tbody></table>
        </div>`;
    }).join('');
    box.innerHTML = `<div class="small muted" style="padding:4px 0">累计失败编译 ${r.total} 次，涉及 ${r.groups.length
    } 个实验</div>${groups}`;
    if (notify) toast('已刷新错误统计', 'ok');
  } catch (e) {
    box.innerHTML = '<div class="empty">加载失败：' + esc(e.message) + '</div>';
  }
}
/* ---------------------------------------------------------------- AI 知识库 */
async function loadKbDocs() {
  const box = $('#kbDocs');
  if (!box) return;
  try {
    const r = await api('/api/kb/docs');
    if (!r.docs.length) {
      box.innerHTML = '<div class="empty">还没有资料。上传讲义或粘贴要点后，AI 回答时会优先参考它们。</div>';
      return;
    }
    const rows = r.docs.map(d => `
      <tr>
        <td>${esc(d.title)}</td>
        <td class="small muted">${esc(d.filename || '—')}</td>
        <td>${d.chars} 字</td>
        <td class="small">${esc(d.created_at || '')}</td>
        <td><button class="btn sm" data-kbdel="${d.id}">删除</button></td>
      </tr>`).join('');
    box.innerHTML = `<div class="small muted" style="padding:4px 0">共 ${r.docs.length} 份资料，合计 ${r.totalChars
    } 字</div>
      <table class="data-table"><thead><tr><th>标题</th><th>文件名</th><th>字数</th><th>上传时间</th><th></th></tr></th
    ead><tbody>${rows}</tbody></table>`;
```

```
  } catch (e) {
    box.innerHTML = '<div class="empty">加载失败：' + esc(e.message) + '</div>';
  }
}
function uploadKbFiles(ev) {
  const files = [...(ev.target.files || [])];
  if (!files.length) return;
  let done = 0, failed = 0;
  const next = (i) => {
    if (i >= files.length) {
      ev.target.value = '';
      toast(`已上传 ${done} 份${failed ? '，失败 ' + failed + ' 份' : ''}`, done ? 'ok' : 'err');
      loadKbDocs();
      return;
    }
    const f = files[i];
    const reader = new FileReader();
    reader.onload = async () => {
      let text = String(reader.result || '');
      if (/\.html?$/i.test(f.name)) {
        text = text.replace(/<script[\s\S]*?<\/script>/gi, ' ').replace(/<style[\s\S]*?<\/style>/gi, ' ')
          .replace(/<[^>]+>/g, ' ').replace(/ /g, ' ').replace(/\s{3,}/g, '\n\n');
      }
      try {
        await api('/api/kb/docs', { method: 'POST', body: { title: f.name.replace(/\.[^.]+$/, ''), filename:
     f.name, content: text } });
        done++;
      } catch { failed++; }
      next(i + 1);
    };
    reader.onerror = () => { failed++; next(i + 1); };
    reader.readAsText(f, 'utf-8');
  };
  next(0);
}
async function pasteKbDoc() {
  const r = await promptBox('粘贴资料', [
    { name: 'title', label: '资料标题', placeholder: '如 第 3 讲 定时器与中断' },
    { name: 'content', label: '内容（可直接从 Word / PDF 复制）', type: 'textarea', placeholder: '把讲义要点粘贴到这里…' }
  ]);
  if (!r || !r.content) return;
  try {
    await api('/api/kb/docs', { method: 'POST', body: { title: r.title || '粘贴的资料', content: r.content } });
    toast('已加入知识库', 'ok');
    loadKbDocs();
  } catch (e) { toast('保存失败：' + e.message, 'err'); }
}
async function deleteKbDoc(id) {
  const ok = await confirmBox('删除资料', '删除后 AI 将不再参考这份资料。确定删除吗？');
  if (!ok) return;
```

```
  try {
    await api(`/api/kb/docs/${id}/delete`, { method: 'POST' });
    toast('已删除', 'ok');
    loadKbDocs();
  } catch (e) { toast('删除失败: ' + e.message, 'err'); }
}
/* ---------------------------------------------------------------- 联网搜索配置 */
async function loadWebSearchCfg() {
  try {
    const c = await api('/api/websearch/config');
    const en = $('#wsEnabled'); if (en) en.checked = !!c.enabled;
    const pv = $('#wsProvider'); if (pv) pv.value = c.provider || 'tavily';
    const mx = $('#wsMax'); if (mx) mx.value = c.maxResults || 5;
    const st = $('#wsState');
    if (st) st.textContent = c.hasKey ? '已配置密钥' : '尚未配置密钥（未配置时 AI 不会联网搜索）';
  } catch { }
}
async function saveWebSearch() {
  try {
    await api('/api/websearch/config', {
      method: 'POST',
      body: {
        enabled: $('#wsEnabled').checked,
        provider: $('#wsProvider').value,
        apiKey: $('#wsKey').value,
        maxResults: Number($('#wsMax').value) || 5
      }
    });
    $('#wsKey').value = '';
    toast('联网搜索设置已保存', 'ok');
    loadWebSearchCfg();
  } catch (e) { toast('保存失败: ' + e.message, 'err'); }
}
/* ---------------------------------------------------------------- 文件导入
 * 支持 CSV / TSV / TXT: 浏览器读成文本后并入导入框, 走同一套解析。
 * PDF / XLSX 是二进制或压缩包格式, 纯前端解析不可靠, 明确提示用户另存为 CSV。
 */
async function onImportFiles(ev) {
  const files = [...(ev.target.files || [])];
  if (!files.length) return;
  const parts = [];
  let failed = 0;
  for (const f of files) {
    try {
      let text = await f.text();
      text = text.replace(/^\uFEFF/, '');      // 去 BOM, Excel 导出的 CSV 常见
      if (!text.trim()) continue;
      parts.push(text.trim());
    } catch { failed++; }
  }
```

```
  const ta = $('#importText');
  const merged = parts.join('\n');
  if (merged) ta.value = (ta.value.trim() ? ta.value.trim() + '\n' : '') + merged;
  const lines = merged.split(/\r?\n/).filter(l => l.trim()).length;
  $('#importFileName').textContent =
    `已读入 ${files.length - failed} 个文件，共 ${lines} 行` + (failed ? `，${failed} 个读取失败` : '');
  ev.target.value = '';
  toast('文件已读入，请确认右侧内容后点「导入」', 'ok');
}
/* ---------------------------------------------------------------- 批量删除 */
async function deleteSelectedStudents() {
  const ids = [...selected];
  if (!ids.length) return toast('请先勾选要删除的学生', 'warn');
  const names = students.filter(s => ids.includes(s.id)).map(s => `${s.name || ''}(${s.student_no || s.usern
    ame})`);
  const okDel = await confirmBox('删除学生', `将删除选中的 ${ids.length} 名学生及其全部记录（练习、成绩、提问），不可恢复：\n\n${name
     8).join('、')}${names.length > 8 ? ' 等' : ''}`);
  if (!okDel) return;
  try {
    const r = await api('/api/teacher/students/delete', { method: 'POST', body: { ids } });
    toast(`已删除 ${r.deleted || ids.length} 名学生`, 'ok');
    selected.clear();
    page = 1;
    await loadStats(true);
  } catch (e) { toast('删除失败：' + e.message, 'err'); }
}
/* ---------------------------------------------------------------- 学情明细
 * 教师视角要回答两个问题：
 *   1) 这个学生具体写了什么、改到哪一版（每次编译都含完整代码）；
 *   2) 这些操作发生在"规定实验时间内"还是课外自主练习。
 * 后端把 attempts 与 assignments 的时间窗对齐后返回。
 */
async function showStudentActivity(id) {
  const st = students.find(s => s.id === id) || {};
  const wrap = el('div', { class: 'md' });
  wrap.innerHTML = '<p class="muted small">加载中…</p>';
  modal(`学情明细 · ${st.name || st.username || ''}`, wrap, [{ label: '关闭', primary: true }]);
  let d;
  try {
    d = await api(`/api/teacher/students/${id}/activity?limit=50`);
  } catch (e) {
    wrap.innerHTML = '<p class="muted">加载失败：' + esc(e.message) + '</p>';
    return;
  }
  const stu = d.student || {};
  const asgs = (d.assignments || []).filter(a => a.startAt || a.endAt || a.openedAt || a.submittedAt);
  const asgRows = asgs.map(a => {
    const state = a.submittedAt ? '已提交' : (a.openedAt ? '进行中' : '未参与');
    return `<tr>
      <td>${esc(a.title)}</td>
```

```
      <td class="small">${esc(a.startAt || '不限')} ~ ${esc(a.endAt || '不限')}</td>
      <td class="small">${a.openedAt ? esc(a.openedAt) : '—'}</td>
      <td class="small">${a.submittedAt ? esc(a.submittedAt) : '—'}</td>
      <td class="small">${a.minutes != null ? a.minutes + ' 分钟' : '—'}</td>
      <td>${state}</td>
    </tr>`;
  }).join('');
  const tl = (d.timeline || []).map((t, i) => {
    const where = t.inWindow
      ? `<span class="tag basic">课内 · ${esc(t.windowTitle)}</span>`
      : '<span class="tag">课外练习</span>';
    const result = t.ok ? '<span class="tag basic">成功</span>' : '<span class="tag" style="background:#fdecea
    ;color:#b3261e">失败</span>';
    const err = !t.ok && t.message ? `<div class="small" style="color:#b3261e;margin-top:2px">${esc(t.messag
    e.slice(0, 200))}</div>` : '';
    return `<div style="border-bottom:1px solid #eef1f5;padding:6px 0">
      <div class="row" style="gap:8px;align-items:center">
        <span class="small muted" style="width:132px">${esc(t.at)}</span>
        <span class="small" style="width:56px">${t.kind === 'compile' ? '编译' : esc(t.kind)}</span>
        ${result}<span class="small muted">${esc(t.problemId || '')}</span>${where}
      </div>
      ${err}
      ${t.code ? `<details><summary class="small muted" style="cursor:pointer">查看当时的代码</summary>
        <pre class="small" style="white-space:pre-wrap;max-height:220px;overflow:auto;margin:4px 0 0">${esc(
    t.code)}</pre></details>` : ''}
    </div>`;
  }).join('') || '<p class="muted small">这个学生还没有编译/仿真记录。</p>';
  wrap.innerHTML = `
    <p class="small muted">${esc(stu.name || '')}  ${esc(stu.studentNo || stu.username || '')}  ${esc(stu.clas
    sName || '')}</p>
    <h4 style="margin:10px 0 4px">作业参与</h4>
    ${asgRows ? `<table class="data-table"><thead><tr><th>任务</th><th>时间窗口</th><th>开始</th><th>提交</th><th>用时</
    th><th>状态</th></tr></thead><tbody>${asgRows}</tbody></table>`
      : '<p class="muted small">暂无实验任务记录。</p>'}
    <h4 style="margin:14px 0 4px">最近操作（含代码快照，最多 50 条）</h4>
    <div>${tl}</div>`;
}
/* ==================== platform/include/intrins.h ==================== */
/*
 * intrins.h — Keil C51 兼容的 intrins.h 替身
 * ============================================================
 * 背景：SDCC 并不提供 <intrins.h>（这是 Keil C51 的头文件），
 *       但大量教材与实验代码（含本平台的起始代码）使用 _crol_/_cror_/_nop_。
 *       为保证学生从 Keil 迁移过来的代码可以直接编译，平台自带本头文件，
 *       并在编译时通过 -I 注入到包含路径中。
 *
 * 说明：
 *   · 旋转函数用纯 C 表达式实现，SDCC 会优化成 RR A / RL A 等单字节指令。
 *   · _nop_() 实现为空语句。真实 Keil 的 _nop_ 会插入 1 个机器周期的空操作，
 *     对本平台的实验（延时/流水灯/数码管扫描）无实质影响；
```

```
 *     若需精确时序，请直接使用 __asm 插入 nop。
 */
#ifndef MCULAB_INTRINS_H
#define MCULAB_INTRINS_H
/* ---------------- 8 位循环移位 ---------------- */
#define _crol_(a, n)  ((unsigned char)(((unsigned char)(a) << (n)) | ((unsigned char)(a) >> (8 - (n)))))
#define _cror_(a, n)  ((unsigned char)(((unsigned char)(a) >> (n)) | ((unsigned char)(a) << (8 - (n)))))
/* ---------------- 16 位循环移位 ---------------- */
#define _irol_(a, n)  ((unsigned int)(((unsigned int)(a) << (n)) | ((unsigned int)(a) >> (16 - (n)))))
#define _iror_(a, n)  ((unsigned int)(((unsigned int)(a) >> (n)) | ((unsigned int)(a) << (16 - (n)))))
/* ---------------- 32 位循环移位 ---------------- */
#define _lrol_(a, n)  ((unsigned long)(((unsigned long)(a) << (n)) | ((unsigned long)(a) >> (32 - (n)))))
#define _lror_(a, n)  ((unsigned long)(((unsigned long)(a) >> (n)) | ((unsigned long)(a) << (32 - (n)))))
/* ---------------- 空操作 / 位测试 ---------------- */
#define _nop_()       ((void)0)
#define _testbit_(b)  ((b) != 0)
#endif /* MCULAB_INTRINS_H */
/* ==================== platform/include/reg51.h ==================== */
/* reg51.h — Keil 兼容垫片：等价于 SDCC 的 <8051.h> */
#ifndef MCULAB_REG51_H
#define MCULAB_REG51_H
#include <8051.h>
#endif
/* ==================== platform/include/reg52.h ==================== */
/* reg52.h — Keil 兼容垫片：等价于 SDCC 的 <8052.h>（含 T2） */
#ifndef MCULAB_REG52_H
#define MCULAB_REG52_H
#include <8052.h>
#endif
/* ==================== platform/include/stc89.h ==================== */
/*--------------------------------------------------------------------------
   stc89c.h - Register Declarations for STC89Cxx Series
   Based on stc12.h
   Copyright (c) 2012, intron@intron.ac
   Copyright (c) 2020, flameeyes@flameeyes.com
   This library is free software; you can redistribute it and/or modify it
   under the terms of the GNU General Public License as published by the
   Free Software Foundation; either version 2, or (at your option) any
   later version.
   This library is distributed in the hope that it will be useful,
   but WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
   GNU General Public License for more details.
   You should have received a copy of the GNU General Public License
   along with this library; see the file COPYING. If not, write to the
   Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston,
   MA 02110-1301, USA.
   As a special exception, if you link this library with other files,
   some of which are compiled with SDCC, to produce an executable,
   this library does not by itself cause the resulting executable to
```

```
   be covered by the GNU General Public License. This exception does
   not however invalidate any other reasons why the executable file
   might be covered by the GNU General Public License.
------------------------------------------------------------------------*/
/*
   Brief:
       STC89Cxx/STC89LExx series are 8051-compatible MCU's. The "official"
   websites are http://www.stcmcu.com/ (In Chinese Han only) and
   http://www.stcmicro.com/ (in English), and datasheets in Chinese Han and
   English can be downloaded there.
   Reference:
   1.
   1. The "official" C header file (written for another C51 compiler):
        http://www.stcmcu.com/datasheet/stc/STC-AD-PDF/STC_NEW_8051.H
   2. Datasheets for STC89(C/LE)xx series:
      English:
        http://www.stcmicro.com/datasheet/STC89C51RC-en.pdf
 */
#ifndef _STC89_H_
#define _STC89_H_
#include <8051.h>
#include <compiler.h>
#ifdef REG8051_H
#undef REG8051_H
#endif
/*
 * Auxiliary Register
 *   Bit Mapping: - - - - - - EXTRAM ALEOFF
 *   Reset Value: xxxx,xx00
 */
SFR(AUXR, 0x8E);
/*
 * Auxiliary Register 1
 *   Bit Mapping: - - - - - - GF2 - - DPS
 *   Reset Value: 0xxx,0xx0
 */
SFR(AUXR1, 0xA2);
/* Slave Address for Serial Communication
 *   Reset Value: 0000,0000
 */
SFR(SADDR, 0xA9);
/*
 * Higher bits for Interrupt Priority
 *   Bit Mapping: PX3H PX2H PT2H PSH PT1H PX1H PT0H PX0H
 *   Reset Value: 0000,0000
 */
SFR(IPH, 0xB7);
/* Slave Address Mask for Serial Communication
 *   Reset Value: 0000,0000
 */
```

```
SFR(SADEN, 0xB9);
/* Auxiliary Input Control
 *   Bit Mapping: PX3 EX3 IE3 IT3 PX2 EX2 HE2 IT2
 *   Reset Value: 0000,0000
 */
SFR(XICON, 0xC0);
/* Timer/Counter 2 registers are compatible with AT89x52 */
/* Timer/Counter 2 Control
 *   Bit Mapping: TF2 EXF2 RCLK TCLK EXEN2 TR2 C/!T2 CP/!RL2
 *   Reset Value: 0000,0000
 */
SFR(T2CON, 0xC8);
/* Timer/Counter 2 Mode
 *   Bit Mapping: - - - - - - T2OE DCEN
 *   Reset Value: xxxx,xx00
 */
SFR(T2MOD, 0xC9);
/* Timer/Counter 2 Reload/Capture Low Byte
 *   Reset Value: 0000,0000
 */
SFR(RCAP2L, 0xCA);
/* Timer/Counter 2 Reload/Capture High Byte
 *   Reset Value: 0000,0000
 */
SFR(RCAP2H, 0xCB);
/* Timer/Counter 2 Low Byte
 *   Reset Value: 0000,0000
 */
SFR(TL2, 0xCC);
/* Timer/Counter 2 High Byte
 *   Reset Value: 0000,0000
 */
SFR(TH2, 0xCD);
/*
 * Watchdog Timer Control Register
 *   Bit Mapping: - - EN_WDT CLR_WDT IDLE_WDT PS2 PS1 PS0
 *   Reset Value: xx00,0000
 */
SFR(WDT_CONTR, 0xE1);
/* In-Application-Programming Data Register
 *   Reset Value: 1111,1111
 */
SFR(IAP_DATA, 0xE2);
/* In-Application-Programming Address Registers
 *   Reset Values: 0000, 0000
 *                 0000, 0000
 */
SFR(IAP_ADDRH, 0xE3); /* Higher 8 bits */
SFR(IAP_ADDRL, 0xE4); /* Lower 8 bits */
/*
```

```
 * In-Application-Programming Address Registers
 *   Bit Mapping: - - - - - MS2 MS1 MS0
 *   Reset Value: xxxx,x000
 */
SFR(IAP_CMD, 0xE5);
/* In-Application-Programming Trigger Registers
 *   Reset Value: xxxx,xxxx
 */
SFR(IAP_TRIG, 0xE6);
/*
 * In-Application-Programming Control Register
 *   Bit Mapping: ISPEN SWBS SWRST - - WT2 WT1 WT0
 *   Reset Value: 000x,x000
 */
SFR(IAP_CONTR, 0xE7);
/*
 * One Extended GPIO Port: P4
 * - For PLCC-44 and LQFP-44 packages, only lower 4 bits of P4 are available.
 */
SFR(P4, 0xE8);
    SBIT(P4_0, 0xC0, 0);
    SBIT(P4_1, 0xC0, 1);
    SBIT(P4_2, 0xC0, 2);
    SBIT(P4_3, 0xC0, 3);
#endif /* _STC89_H_ */
/* ==================== tools/calibrate_delay.mjs ==================== */
/**
 * calibrate_delay.mjs — 标定软件延时循环的常数
 * 目标：delay_ms(1) ≈ 921.6 个机器周期（11.0592MHz / 12T）
 */
import { Compiler } from '../server/compile.js';
import { CPU8051 } from '../public/js/core/cpu8051.js';
const compiler = new Compiler();
const probe = await compiler.probe();
if (!probe.ok) { console.error('SDCC 不可用：', probe.error); process.exit(2); }
async function measure(inner) {
  const src = `
#include <stc89.h>
void delay_ms(unsigned int ms){
    volatile unsigned int i, j;
    for(i = 0; i < ms; i++)
        for(j = 0; j < ${inner}; j++);
}
void main(void){
    P1 = 0x00;
    delay_ms(100);
    P1 = 0xFF;
    while(1);
}
`;
```

```
  const r = await compiler.compile(src, { name: 'cal' + inner });
  if (!r.ok) throw new Error(r.errors?.[0]?.msg || 'compile failed');
  const cpu = new CPU8051();
  cpu.loadHex(r.hex);
  // 先等到 P1=0x00（延时开始），再等到 P1=0xFF（延时结束）
  let guard = 0, t0 = null;
  // 注意：step() 内部已经调用 tick() 推进外设与 cycles，
  //       外部再调 tick() 会把时间记账翻倍（这正是早期标定偏小一半的原因）。
  while (guard++ < 60_000_000) {
    cpu.step();
    const p1 = cpu.readPort(1);
    if (t0 === null && p1 === 0x00) t0 = cpu.cycles;
    else if (t0 !== null && p1 === 0xFF) break;
  }
  if (t0 === null) throw new Error('未捕捉到延时起点');
  return (cpu.cycles - t0) / 100;   // 每 1ms 的机器周期数
}
for (const inner of [26, 27, 28, 30, 32]) {
  const perMs = await measure(inner);
  console.log(`inner=${String(inner).padStart(3)}  →  delay_ms(1) ≈ ${perMs.toFixed(1)} 机器周期  (目标 921.6, 误差
    ${((perMs/921.6-1)*100).toFixed(1)}%)`);
}
/* ==================== tools/check_dom_contract.mjs ==================== */
/**
 * check_dom_contract.mjs — 前后端 DOM 契约静态检查
 * 扫描 public/js/ui/*.js 与 app.js 中所有 $('#id') / document.getElementById('id')，
 * 与 public/index.html 中的 id 集合比对，找出选择器写错/漏写的元素。
 * 这类错误在 Node 里不会报，只有浏览器里才会 undefined，所以单列一个检查。
 */
import fs from 'node:fs';
import path from 'node:path';
const ROOT = path.resolve(import.meta.dirname, '..');
const html = fs.readFileSync(path.join(ROOT, 'public/index.html'), 'utf8');
const ids = new Set([...html.matchAll(/\bid="([^"]+)"/g)].map(m => m[1]));
// 由 JS 在模态框里动态创建的元素（不在 index.html 中），以及形如 '#view-' 的模板拼接
const DYNAMIC = new Set(['pwdOld', 'pwdNew', 'pwdNew2', 'tAns', 'tAnsSend', 'tAnsClose', 'netBarClose', 'sta
    geSelect', 'btnAssignStart', 'btnAssignSubmit', 'schState', 'openCode', 'codeState', 'openReport', 'subs
    , 'openSchInEditor']);
const isDynamicPattern = (id) => id === 'view-' || id.endsWith('-');
const files = ['public/js/app.js', ...fs.readdirSync(path.join(ROOT, 'public/js/ui')).map(f => 'public/js/ui
    /' + f)];
let bad = 0, checked = 0;
const report = [];
for (const rel of files) {
  const src = fs.readFileSync(path.join(ROOT, rel), 'utf8');
  const refs = new Set();
  for (const m of src.matchAll(/\$\(\s*'#([A-Za-z0-9_-]+)'/g)) refs.add(m[1]);
  for (const m of src.matchAll(/getElementById\(\s*'([A-Za-z0-9_-]+)'/g)) refs.add(m[1]);
  const missing = [...refs].filter(id => !ids.has(id) && !DYNAMIC.has(id) && !isDynamicPattern(id));
  checked += refs.size;
```

```
  if (missing.length) { bad += missing.length; report.push(`  ✗ ${rel}: 找不到 #${missing.join(', #')}`); }
}
// 反向检查：HTML 里定义了但从未被引用的 id（仅提示）
const used = new Set();
for (const rel of files) {
  const src = fs.readFileSync(path.join(ROOT, rel), 'utf8');
  for (const m of src.matchAll(/['"]#([A-Za-z0-9_-]+)['"]/g)) used.add(m[1]);
  for (const m of src.matchAll(/getElementById\(\s*'([A-Za-z0-9_-]+)'/g)) used.add(m[1]);
}
const unused = [...ids].filter(id => !used.has(id));
console.log('\n===== DOM 契约检查 =====');
console.log(`index.html 定义 ${ids.size} 个 id；JS 引用 ${checked} 处`);
console.log(bad ? report.join('\n') : '  ✓ 所有 JS 选择器都能在 index.html 中找到');
console.log(`\n未被 JS 直接引用的 id（${unused.length} 个，通常是 CSS 或纯展示用）：${unused.slice(0, 25).join(', ')}`);
console.log(`\n结果：${bad === 0 ? '通过' : bad + ' 处缺失'}`);
process.exit(bad ? 1 : 0);
/* ==================== tools/extract_experiments.mjs ==================== */
/** 从 server/seed.js 提取每个实验的起始代码，写为独立 .c 文件（便于核查与编译） */
import fs from 'node:fs';
import path from 'node:path';
import { PROBLEMS } from '../server/seed.js';
const ROOT = path.resolve(import.meta.dirname, '..');
const DIR = path.join(ROOT, 'data', 'experiments');
fs.mkdirSync(DIR, { recursive: true });
let n = 0;
for (const p of PROBLEMS) {
  if (!p.starter) continue;
  fs.writeFileSync(path.join(DIR, p.id + '.c'), p.starter, 'utf8');
  n++;
}
console.log(`已导出 ${n} 个实验源码到 data/experiments/`);
/* ==================== tools/make_cloud_bundle.mjs ==================== */
/**
 * make_cloud_bundle.mjs — 生成"云端发布包"
 * ================================================================
 * 为什么不直接发布项目根目录：
 *   · toolchain/sdcc 是 280MB 的 Windows 版，云端用不上，上传会极慢甚至失败；
 *   · data/ 里有学生数据库与编译临时文件，不该上传；
 *   · tests/ docs/ certs/ .playwright-cli/ 与运行无关。
 *
 * 产出：cloud/（约 2MB），供发布工具上传。
 * 用法：node tools/make_cloud_bundle.mjs
 */
import fs from 'node:fs';
import path from 'node:path';
const ROOT = path.resolve(import.meta.dirname, '..');
const OUT = path.join(ROOT, 'cloud');
const INCLUDE = [
  'package.json',
  'README.md',
```

```
  'server',
  'public',
  'platform',
  'tools'
];
function copyRec(src, dst) {
  const st = fs.statSync(src);
  if (st.isDirectory()) {
    fs.mkdirSync(dst, { recursive: true });
    for (const name of fs.readdirSync(src)) copyRec(path.join(src, name), path.join(dst, name));
  } else {
    fs.copyFileSync(src, dst);
  }
}
fs.rmSync(OUT, { recursive: true, force: true });
fs.mkdirSync(OUT, { recursive: true });
let files = 0, bytes = 0;
for (const rel of INCLUDE) {
  const src = path.join(ROOT, rel);
  if (!fs.existsSync(src)) { console.warn('  跳过（不存在）:', rel); continue; }
  const dst = path.join(OUT, rel);
  copyRec(src, dst);
  const walk = (p) => {
    const s = fs.statSync(p);
    if (s.isDirectory()) for (const n of fs.readdirSync(p)) walk(path.join(p, n));
    else { files++; bytes += s.size; }
  };
  walk(dst);
}
// 云端必须自带 Linux 版 SDCC：沙箱里没有 apt、也装不了包，不带上就没法编译
const LINUX_SDCC = path.join(ROOT, 'toolchain', 'sdcc-linux');
if (fs.existsSync(LINUX_SDCC)) {
  const dst = path.join(OUT, 'toolchain', 'sdcc-linux');
  copyRec(LINUX_SDCC, dst);
  const walk2 = (p) => {
    const st = fs.statSync(p);
    if (st.isDirectory()) for (const n of fs.readdirSync(p)) walk2(path.join(p, n));
    else { files++; bytes += st.size; }
  };
  walk2(dst);
  console.log('  ✓ 已包含 Linux 版 SDCC（toolchain/sdcc-linux）');
} else {
  console.warn('  ! 未找到 toolchain/sdcc-linux，云端将无法编译 C');
}
// 云端需要可写的数据目录占位
fs.mkdirSync(path.join(OUT, 'data'), { recursive: true });
console.log(`  ✓ cloud/ 已生成：${files} 个文件，${(bytes / 1024 / 1024).toFixed(2)} MB`);
console.log('  下一步：用发布工具发布 cloud/ 目录');
/* ==================== tools/make_icons.mjs ==================== */
/**
```

```
 * make_icons.mjs — 生成 PWA 图标（纯 Node，无第三方依赖）
 * ------------------------------------------------------------------
 * 产出 public/icons/icon-192.png、icon-512.png
 * 图形：深蓝圆角方块 + 白色"芯片引脚"符号，与界面左上角 MCU 标识一致。
 * 这样学生可以把平台"添加到主屏幕"，弱网时点开即用（配合 Service Worker 离线缓存）。
 */
import zlib from 'node:zlib';
import fs from 'node:fs';
import path from 'node:path';
const OUT = path.resolve(import.meta.dirname, '..', 'public', 'icons');
fs.mkdirSync(OUT, { recursive: true });
/* ---------------- 最小 PNG 编码器 ---------------- */
const crcTable = (() => {
  const t = new Int32Array(256);
  for (let n = 0; n < 256; n++) {
    let c = n;
    for (let k = 0; k < 8; k++) c = c & 1 ? 0xEDB88320 ^ (c >>> 1) : c >>> 1;
    t[n] = c;
  }
  return t;
})();
function crc32(buf) {
  let c = -1;
  for (let i = 0; i < buf.length; i++) c = crcTable[(c ^ buf[i]) & 0xFF] ^ (c >>> 8);
  return (c ^ -1) >>> 0;
}
function chunk(type, data) {
  const len = Buffer.alloc(4);
  len.writeUInt32BE(data.length);
  const body = Buffer.concat([Buffer.from(type, 'ascii'), data]);
  const crc = Buffer.alloc(4);
  crc.writeUInt32BE(crc32(body));
  return Buffer.concat([len, body, crc]);
}
function encodePNG(w, h, rgba) {
  const sig = Buffer.from([0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A]);
  const ihdr = Buffer.alloc(13);
  ihdr.writeUInt32BE(w, 0); ihdr.writeUInt32BE(h, 4);
  ihdr[8] = 8; ihdr[9] = 6; ihdr[10] = 0; ihdr[11] = 0; ihdr[12] = 0;
  // 每行前加 filter 字节 0
  const raw = Buffer.alloc((w * 4 + 1) * h);
  for (let y = 0; y < h; y++) {
    raw[y * (w * 4 + 1)] = 0;
    rgba.copy(raw, y * (w * 4 + 1) + 1, y * w * 4, (y + 1) * w * 4);
  }
  return Buffer.concat([sig, chunk('IHDR', ihdr), chunk('IDAT', zlib.deflateSync(raw, { level: 9 })), chunk(
    'IEND', Buffer.alloc(0))]);
}
/* ---------------- 绘制 ---------------- */
function draw(size) {
```

```
const px = Buffer.alloc(size * size * 4);
const set = (x, y, r, g, b, a = 255) => {
  if (x < 0 || y < 0 || x >= size || y >= size) return;
  const i = (y * size + x) * 4;
  px[i] = r; px[i + 1] = g; px[i + 2] = b; px[i + 3] = a;
};
// 圆角半径
const R = Math.round(size * 0.22);
const insideRound = (x, y) => {
  const cx = Math.min(Math.max(x, R), size - 1 - R);
  const cy = Math.min(Math.max(y, R), size - 1 - R);
  const dx = x - cx, dy = y - cy;
  return dx * dx + dy * dy <= R * R;
};
// 背景：对角渐变（左上较亮 → 右下较深），带 1px 抗锯齿
for (let y = 0; y < size; y++) {
  for (let x = 0; x < size; x++) {
    if (!insideRound(x, y)) { set(x, y, 0, 0, 0, 0); continue; }
    const t = (x + y) / (2 * size);
    const r = Math.round(27 + (10 - 27) * t);
    const g = Math.round(99 + (52 - 99) * t);
    const b = Math.round(214 + (121 - 214) * t);
    set(x, y, r, g, b, 255);
  }
}
// 芯片本体（白色圆角矩形）
const m = Math.round(size * 0.28);          // 边距
const cw = size - m * 2, chh = cw;
const rad = Math.round(size * 0.06);
const inChip = (x, y) => {
  const x0 = m, y0 = m, x1 = m + cw, y1 = m + chh;
  if (x < x0 || y < y0 || x >= x1 || y >= y1) return false;
  const cx = Math.min(Math.max(x, x0 + rad), x1 - 1 - rad);
  const cy = Math.min(Math.max(y, y0 + rad), y1 - 1 - rad);
  const dx = x - cx, dy = y - cy;
  return dx * dx + dy * dy <= rad * rad;
};
const pinW = Math.max(2, Math.round(size * 0.030));
const pinLen = Math.round(size * 0.085);
for (let y = 0; y < size; y++) {
  for (let x = 0; x < size; x++) {
    if (inChip(x, y)) set(x, y, 255, 255, 255, 255);
  }
}
// 引脚：四边各 4 根
const n = 4;
const span = cw * 0.62, off = (cw - span) / 2;
for (let i = 0; i < n; i++) {
  const p = Math.round(off + (span * i) / (n - 1));
  for (let t = 0; t < pinLen; t++) {
```

```
      // 左/右
      for (let w = 0; w < pinW; w++) {
        set(m - t, m + p + w, 255, 255, 255, t < pinLen - 1 ? 255 : 200);
        set(m + cw + t, m + p + w, 255, 255, 255, t < pinLen - 1 ? 255 : 200);
      }
      // 上/下
      for (let w = 0; w < pinW; w++) {
        set(m + p + w, m - t, 255, 255, 255, t < pinLen - 1 ? 255 : 200);
        set(m + p + w, m + chh + t, 255, 255, 255, t < pinLen - 1 ? 255 : 200);
      }
    }
  }
  // 中央一个深色圆点，像芯片的定位标记
  const cc = m + cw / 2, rr = size * 0.045;
  for (let y = 0; y < size; y++) {
    for (let x = 0; x < size; x++) {
      const dx = x - cc, dy = y - cc;
      if (dx * dx + dy * dy <= rr * rr) set(x, y, 24, 90, 189, 255);
    }
  }
  return px;
}
for (const size of [192, 512]) {
  const png = encodePNG(size, size, draw(size));
  const f = path.join(OUT, `icon-${size}.png`);
  fs.writeFileSync(f, png);
  console.log(`  ✓ ${path.relative(process.cwd(), f)}  ${png.length} 字节`);
}
console.log('图标生成完成');
/* ==================== tools/make_sw_docs.mjs ==================== */
/**
 * make_sw_docs.mjs — 生成软件著作权登记用的"源程序"与"用户手册"
 * ---------------------------------------------------------------
 * 软著对源程序材料的要求（形式审查，卡得很死）：
 *   · 前 30 页 + 后 30 页，共 60 页；不足 60 页则全部提交
 *   · 每页 50 行，连续排版，不能有空白页
 *   · 页眉需含：软件名称 + 版本号 + 页码
 *
 * 实现方式：直接生成 A4 固定尺寸的 HTML 页（每页一个 div），
 * 再用 Edge 无头模式打印成 PDF。这样分页、页眉、页码都可精确控制，
 * 不依赖任何 npm 包，也不受系统中文字体缺失影响。
 */
import fs from 'node:fs';
import path from 'node:path';
const ROOT = path.resolve(import.meta.dirname, '..');
const OUT = path.join(ROOT, 'build', 'sw');
const SOFT_NAME = '微控制器教学仿真实验平台';
const VERSION = 'V1.0';
const LINES_PER_PAGE = 50;
const MAX_COLS = 108;          // 单行最大字符数（超出折行，保证每页 50 行不变）
```

```
const PAGES = 60;              // 前 30 + 后 30
const esc = (s) => s.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
/* ---------------- 1. 收集源程序 ---------------- */
function collectFiles() {
  const groups = [
    ['server', (f) => f.endsWith('.js')],
    ['public/js', (f) => f.endsWith('.js') && !f.includes('vendor')],
    ['platform', (f) => f.endsWith('.js') || f.endsWith('.h')],
    ['tools', (f) => f.endsWith('.mjs') || f.endsWith('.js')]
  ];
  const files = [];
  let buffer = [];
  for (const [dir, ok] of groups) {
    const abs = path.join(ROOT, dir);
    if (!fs.existsSync(abs)) continue;
    const walk = (d) => {
      for (const e of fs.readdirSync(d, { withFileTypes: true })) {
        const p = path.join(d, e.name);
        if (e.isDirectory()) { if (e.name !== 'node_modules' && e.name !== 'vendor') walk(p); }
        else if (ok(e.name)) buffer.push(p);
      }
    };
    walk(abs);
    // 组内按路径排序，组间保持 groups 定义的顺序:
    // 源程序材料通常要求以"主程序"开头，服务端入口在前更符合审查习惯。
    files.push(...buffer.sort((a, b) => {
      // 入口文件排最前，让源程序材料从"主程序"开始（审查惯例）
      const rank = (f) => (/server[\\/]index\.js$/.test(f) ? 0 : /server[\\/]/.test(f) ? 1 : 2);
      const d = rank(a) - rank(b);
      return d !== 0 ? d : a.localeCompare(b);
    }));
    buffer = [];
  }
  return files;
}
function buildLines() {
  const files = collectFiles();
  const lines = [];
  for (const f of files) {
    const rel = path.relative(ROOT, f).replace(/\\/g, '/');
    lines.push(`/* ==================== ${rel} ==================== */`);
    const raw = fs.readFileSync(f, 'utf8').replace(/\r\n/g, '\n').split('\n');
    for (const ln of raw) {
      if (!ln.trim()) continue;                      // 去掉空行，提高信息密度
      // 超长行折行，折行后仍按"行"计数，保证每页严格 50 行
      for (let i = 0; i < ln.length; i += MAX_COLS) {
        lines.push(i === 0 ? ln.slice(0, MAX_COLS) : '    ' + ln.slice(i, i + MAX_COLS - 4));
      }
    }
  }
```

```
  return lines;
}
/* ---------------- 2. 生成源程序 HTML ---------------- */
function htmlSource(lines) {
  const total = lines.length;
  const half = PAGES / 2 * LINES_PER_PAGE;       // 1500
  let body = '';
  const pages = [];
  if (total <= PAGES * LINES_PER_PAGE) {
    // 不足 60 页：全部提交
    for (let i = 0; i < total; i += LINES_PER_PAGE) pages.push(lines.slice(i, i + LINES_PER_PAGE));
  } else {
    for (let p = 0; p < PAGES / 2; p++) pages.push(lines.slice(p * LINES_PER_PAGE, (p + 1) * LINES_PER_PAGE)
    );
    const tailStart = total - half;
    for (let p = 0; p < PAGES / 2; p++) pages.push(lines.slice(tailStart + p * LINES_PER_PAGE, tailStart + (
    p + 1) * LINES_PER_PAGE));
  }
  pages.forEach((pg, idx) => {
    body += `<div class="page">
  <div class="hd"><span>${SOFT_NAME} ${VERSION}</span><span>源程序</span><span>第 ${idx + 1} 页 / 共 ${pages.lengt
    h} 页</span></div>
  <div class="code">${pg.map(l => `<div class="ln">${esc(l) || ' '}</div>`).join('')}</div>
  <div class="ft">第 ${idx + 1} 页</div>
</div>`;
  });
  return `<!doctype html><html lang="zh-CN"><meta charset="utf-8"><title>${SOFT_NAME} ${VERSION} 源程序</title>
<style>
  @page { size: A4; margin: 0; }
  * { box-sizing: border-box; }
  body { margin: 0; font-family: Consolas, "DengXian", "Microsoft YaHei", monospace; }
  .page {
    width: 210mm; height: 297mm; padding: 12mm 12mm 10mm;
    page-break-after: always; position: relative; background: #fff;
  }
  .page:last-child { page-break-after: auto; }
  .hd {
    display: flex; justify-content: space-between; align-items: baseline;
    font-size: 8pt; color: #333; border-bottom: 0.6pt solid #666; padding-bottom: 2mm; margin-bottom: 3mm;
    font-family: "DengXian", "Microsoft YaHei", sans-serif;
  }
  .code { font-size: 7.6pt; line-height: 1.45; }
  .ln { white-space: pre; height: 4.66mm; overflow: hidden; }
  .ft { position: absolute; left: 0; right: 0; bottom: 6mm; text-align: center; font-size: 8pt; color: #666;
        font-family: "DengXian", "Microsoft YaHei", sans-serif; }
</style><body>${body}</body></html>`;
}
/* ---------------- 3. 生成用户手册 HTML ---------------- */
function htmlManual(shots) {
  const img = (file, cap) => {
```

```
    const p = path.join(ROOT, file);
    if (!fs.existsSync(p)) return '';
    const b64 = fs.readFileSync(p).toString('base64');
    return `<figure><img src="${b64}"><figcaption>${cap}</figcaption></figure>`;
  };
  const page = (inner, no) => `<div class="page">
  <div class="hd"><span>${SOFT_NAME} ${VERSION}</span><span>用户手册</span><span>第 ${no} 页</span></div>
  <div class="bd">${inner}</div>
  <div class="ft">${SOFT_NAME} ${VERSION}  用户手册  第 ${no} 页</div>
</div>`;
  const pages = [];
  pages.push(`<h1>${SOFT_NAME}</h1><h2 style="border:none;margin-top:4mm">用户手册</h2>
    <p class="meta">版本号：${VERSION}</p>
    <p class="meta">文档用途：软件著作权登记 · 软件说明书</p>
    <p class="note">本手册介绍该平台的功能组成、运行环境与操作流程。</p>`);
  pages.push(`<h2>一、软件概述</h2>
    <p>${SOFT_NAME}是面向高校《微控制器原理及应用》（8051 / STC89C52）课程的在线实验教学平台。平台把"读题–编写代码–在线编译–绘制原理图–条实验链路放在浏览器中完成，学生只需一台能上网的电脑，教师可集中管理班级、布置任务与评定成绩。</p>
    <h3>1.1 主要特点</h3>
    <ul>
      <li>在线编译：服务端集成 SDCC 编译器，浏览器内提交源码即产出 Intel HEX，无需学生安装工具链。</li>
      <li>电路仿真：内置 8051 指令级仿真内核与常用外设模型（LED、数码管、LCD1602、矩阵键盘、74HC595 等），仿真在浏览器本地执行。</li>
      <li>原理图编辑：支持放置元件、连线、拖动、旋转、参数编辑与电气规则检查。</li>
      <li>虚拟仪器：逻辑分析仪、示波器、串口助手，用于观察端口时序与通信数据。</li>
      <li>实验报告：结合真实代码、原理图与波形统计自动起草报告草稿。</li>
      <li>教学管理：班级与学生名单管理、实验任务发布、成绩册、开放性命题、AI 知识库。</li>
    </ul>`);
  pages.push(`<h2>二、运行环境</h2>
    <table>
      <tr><th>项目</th><th>要求</th></tr>
      <tr><td>服务端操作系统</td><td>Windows 10/11、Ubuntu 20.04 及以上、Debian 系 Linux</td></tr>
      <tr><td>运行时</td><td>Node.js 22 及以上（使用内置 node:sqlite，无需额外数据库）</td></tr>
      <tr><td>C 编译器</td><td>SDCC 4.x（Windows 便携版随程序分发；Linux 由系统包或镜像内置）</td></tr>
      <tr><td>客户端浏览器</td><td>Chrome 110+ / Edge 110+ / Firefox 110+（需支持 ES Module 与 SVG）</td></tr>
      <tr><td>网络</td><td>局域网或公网均可；单机部署时端口默认 8080</td></tr>
      <tr><td>硬件建议</td><td>服务端 2 核 2GB；客户端无特殊要求</td></tr>
    </table>
    <h3>2.1 启动方式</h3>
    <p>开发或本机部署：在程序目录执行 <code>node server/index.js --port 8080</code>，浏览器访问 <code>http://127.0.0.1:8080</code>。</p>
    <p>容器部署：使用随程序提供的 Dockerfile 构建镜像后运行，数据目录挂载至 <code>/app/data</code> 以持久保存。</p>`);
  pages.push(`<h2>三、登录与账号</h2>
    <p>平台区分教师与学生两种角色。首次部署后自动创建教师账号（用户名 teacher），初始密码由部署时设定。</p>
    <h3>3.1 学生注册</h3>
    <p>学生使用教师导入名单中的学号注册；若教师关闭了自助注册，则由教师统一创建。</p>
    <h3>3.2 教师注册</h3>
    <p>教师注册需要邀请码。已有教师在「教师后台 → 教师邀请码」中查看并分享；首位教师自动获得初始邀请码。</p>
    ${img('.playwright-cli/render-live.png', '图 3-1 登录界面（含登录 / 学生注册 / 教师注册三个页签）')}`);
  pages.push(`<h2>四、实验题目与任务</h2>
    <p>左侧为实验题目列表，可按关键字检索；右侧显示题目的实验目的、接线要求、预期现象与评分要点。</p>
```

```
  <p>教师发布实验任务后，题目详情上方会出现任务横幅，显示实验要求、开放时间与当前状态，并提供「开始实验」与「提交本次实验」两个按钮。系统
  ${img('.playwright-cli/stu-assign.png', '图 4-1 学生端实验任务横幅')}`);
pages.push(`<h2>五、代码编辑与在线编译</h2>
  <p>在「代码 IDE」中编写 C 语言程序，编辑器提供语法高亮与行号。点击编译后，服务端调用 SDCC 生成 HEX，并在下方输出编译信息、代码空间与
  <p>编译失败时给出文件名、行号与错误原因，便于定位问题。若部署环境缺少 SDCC，平台自动启用内置汇编通路以降级运行。</p>
  <h3>5.1 起始代码</h3>
  <p>每个实验题目都附带起始代码，便于学生从可运行的最小框架开始修改。</p>`);
pages.push(`<h2>六、原理图编辑</h2>
  <p>左侧为元件库（按类别分组，可搜索），中间为画布，右侧为属性与诊断信息。</p>
  <h3>6.1 基本操作</h3>
  <ul>
    <li>放置元件：点击元件库条目后，在画布空白处单击。</li>
    <li>连线：切换到「连线」工具，依次点击两个引脚。</li>
    <li>移动：用「选择」工具拖动元件，与之相连的导线会自动跟随。</li>
    <li>旋转：选中元件后点击「旋转」或按 R 键。</li>
    <li>删除：切换到「删除」工具后点击元件或导线。</li>
    <li>参数编辑：双击元件（或点「属性」按钮）修改阻值、容值、标注文字等。</li>
  </ul>
  <p>拖动元件时导线端点始终吸附在引脚上，并对两个引脚不在同一行列的情况自动生成直角拐点。</p>
  ${img('.playwright-cli/wire-follow.png', '图 6-1 原理图编辑：元件、连线与诊断')}`);
pages.push(`<h2>七、仿真运行与仪器</h2>
  <p>点击「仿真运行」后，平台在浏览器本地执行 8051 指令级仿真，按机器周期推进，并刷新外设状态（LED 亮灭、数码管显示、LCD 字符等）。</p
  <h3>7.1 虚拟仪器</h3>
  <ul>
    <li>逻辑分析仪：同时采集多路引脚电平，按时间轴绘制波形，可测量脉宽与周期。</li>
    <li>示波器：观察模拟量变化趋势。</li>
    <li>串口助手：收发串口数据，验证波特率与帧格式。</li>
  </ul>
  <p>仿真停止后可单步执行、设置断点，便于分析程序流程。</p>`);
pages.push(`<h2>八、实验报告</h2>
  <p>「实验报告」页提供结构化报告编辑，包含实验目的、原理、硬件电路、软件设计、结果分析、问题与解决、心得体会等章节。</p>
  <h3>8.1 自动起草</h3>
  <p>点击「自动生成」后，平台把当前代码、原理图要点与波形统计作为依据，调用大模型起草报告草稿；模型不可用时退化为规则模板，保证学生始终能
  <p>生成结果是<strong>草稿</strong>，学生需结合本人实际调试过程修改，「心得体会」保留为占位由学生本人填写。</p>`);
pages.push(`<h2>九、教师后台</h2>
  <p>教师后台以纵向滚动页面组织各功能模块，包括班级与学情、学生名单、AI 知识库、联网搜索、高频错误、学情分析、批量导入、大模型设置、邀请
  <h3>9.1 学生名单</h3>
  <p>名单支持分页浏览（每页 20 人），可全选当前筛选下的全部学生，并进行批量删除。导入支持 CSV / TSV / TXT 文件或直接粘贴，列名可用：学
  <h3>9.2 学情明细</h3>
  <p>点击学生行的「详情」，可查看该生最近的编译/仿真记录（含当时的完整代码）、错误信息，以及每条记录是发生在教师设定的实验时间内（课内）
  ${img('.playwright-cli/teacher-layout.png', '图 9-1 教师后台（班级与学情、导入、知识库等模块纵向排列）')}`);
pages.push(`<h2>十、教学管理功能</h2>
  <h3>10.1 实验任务</h3>
  <p>为指定实验发布任务，设定实验要求与起止时间；系统记录学生打开与提交时间，超出窗口部分不计入用时。</p>
  <h3>10.2 成绩册</h3>
  <p>教师自定义成绩构成：环节（实验或考核项）带权重，环节内可设「功能实现 / 电路设计 / 实验报告 / 调试过程」等子项并各自带权重。总分按权
  >
  ${img('.playwright-cli/grade2.png', '图 10-1 成绩册：左侧成绩构成，右侧逐项打分与总评')}
  <h3>10.3 开放性命题</h3>
  <p>教师自由出题，学生提交原理图、代码与报告，教师给总分并写评语。</p>
```

```
    <h3>10.4 AI 知识库与联网搜索</h3>
    <p>教师可上传讲义文本或粘贴内容，AI 答疑时先检索知识库片段再作答；联网搜索可接入搜索服务，未配置时明确提示不可用。</p>`);
  pages.push(`<h2>十一、数据与安全</h2>
    <ul>
      <li>密码采用加盐哈希存储，服务端不保存明文；接口密钥仅存于服务端数据库，不下发浏览器。</li>
      <li>登录会话使用 HttpOnly Cookie；同一标签页刷新保持登录，关闭浏览器后需重新登录。</li>
      <li>学生数据存放在服务端 SQLite 数据库，部署时建议将数据目录挂载为持久卷并定期备份。</li>
      <li>教师账号拥有班级与学生数据的完整权限，公用电脑使用后应点击「退出」。</li>
    </ul>
    <h2>十二、常见问题</h2>
    <table>
      <tr><th>现象</th><th>处理</th></tr>
      <tr><td>编译提示找不到 SDCC</td><td>确认部署环境已安装 SDCC 或使用随附便携版；Linux 镜像中由构建步骤安装</td></tr>
      <tr><td>AI 回答为"离线助教"</td><td>教师后台未启用大模型或密钥无效，按提示配置后点「测试连接」</td></tr>
      <tr><td>刷新后需重新登录</td><td>会话为标签页级，属预期行为；同标签页刷新不会掉线</td></tr>
      <tr><td>仿真不动</td><td>确认已成功编译并载入 HEX，且晶振与复位电路已连接</td></tr>
    </table>
    <p class="note">—— 手册完 ——</p>`);
  return `<!doctype html><html lang="zh-CN"><meta charset="utf-8"><title>${SOFT_NAME} ${VERSION} 用户手册</title
    >
<style>
  @page { size: A4; margin: 0; }
  * { box-sizing: border-box; }
  body { margin: 0; font-family: "DengXian", "Microsoft YaHei", sans-serif; color: #1a1a1a; }
  .page { width: 210mm; height: 297mm; padding: 14mm 16mm 12mm; page-break-after: always; position: relative
    ; }
  .page:last-child { page-break-after: auto; }
  .hd { display: flex; justify-content: space-between; font-size: 8pt; color: #555;
        border-bottom: 0.6pt solid #999; padding-bottom: 2mm; margin-bottom: 5mm; }
  .ft { position: absolute; left: 0; right: 0; bottom: 7mm; text-align: center; font-size: 8pt; color: #777;
     }
  h1 { font-size: 22pt; text-align: center; margin: 42mm 0 2mm; letter-spacing: 2pt; }
  h2 { font-size: 13pt; margin: 0 0 3mm; padding-bottom: 1.5mm; border-bottom: 1pt solid #333; }
  h3 { font-size: 10.5pt; margin: 4mm 0 1.5mm; color: #16548a; }
  p { font-size: 9.8pt; line-height: 1.75; margin: 0 0 2.4mm; text-align: justify; }
  ul { font-size: 9.8pt; line-height: 1.75; margin: 0 0 2.4mm; padding-left: 6mm; }
  li { margin-bottom: 1mm; }
  code { font-family: Consolas, monospace; background: #f2f4f7; padding: 0 1mm; border-radius: 1mm; font-siz
    e: 9pt; }
  table { width: 100%; border-collapse: collapse; font-size: 9.2pt; margin: 2mm 0 3mm; }
  th, td { border: 0.6pt solid #bbb; padding: 1.8mm 2mm; text-align: left; }
  th { background: #f0f3f7; font-weight: 600; }
  figure { margin: 3mm 0 2mm; text-align: center; }
  figure img { max-width: 100%; max-height: 150mm; border: 0.6pt solid #ccc; border-radius: 1.5mm; }
  figcaption { font-size: 8.5pt; color: #666; margin-top: 1.5mm; }
  .meta { font-size: 10pt; text-align: center; color: #333; }
  .note { font-size: 9pt; color: #666; text-align: center; margin-top: 6mm; }
</style><body>${pages.map((p, i) => page(p, i + 1)).join('')}</body></html>`;
}
/* ---------------- 4. 输出 ---------------- */
```

```
fs.mkdirSync(OUT, { recursive: true });
const lines = buildLines();
const srcHtml = path.join(OUT, 'source.html');
const manHtml = path.join(OUT, 'manual.html');
fs.writeFileSync(srcHtml, htmlSource(lines), 'utf8');
fs.writeFileSync(manHtml, htmlManual(), 'utf8');
console.log(`源程序行数: ${lines.length}`);
console.log(`已生成: ${path.relative(ROOT, srcHtml)}`);
console.log(`已生成: ${path.relative(ROOT, manHtml)}`);
/* ==================== tools/precompile.mjs ==================== */
/**
 * precompile.mjs — 用平台的编译服务预编译全部内置实验
 * ------------------------------------------------------------------
 * 产物: public/hex/<id>.hex       学生即使没有 SDCC 也能直接加载仿真
 *       public/hex/<id>.build.log 编译日志
 *       public/hex/<id>.rst       符号表（可选，供反汇编显示）
 *
 * 为什么用 Node 而不是 bash 直接调 SDCC:
 *   平台的编译服务还包含 Keil→SDCC 语法翻译、intrins.h 注入、printf stub 等,
 *   必须走同一条链路，否则"预编译产物"与"学生实际编译结果"可能不一致。
 */
import fs from 'node:fs';
import path from 'node:path';
import { Compiler } from '../server/compile.js';
const ROOT = path.resolve(import.meta.dirname, '..');
const SRC = path.join(ROOT, 'data', 'experiments');
const OUT = path.join(ROOT, 'public', 'hex');
fs.mkdirSync(OUT, { recursive: true });
const compiler = new Compiler();
const probe = await compiler.probe();
console.log('SDCC:', probe.ok ? (probe.version || 'OK') : ('不可用 — ' + probe.error));
if (!probe.ok) process.exit(2);
const files = fs.readdirSync(SRC).filter(f => f.endsWith('.c')).sort();
let ok = 0, bad = 0;
const rows = [];
for (const f of files) {
  const id = f.replace(/\.c$/, '');
  const code = fs.readFileSync(path.join(SRC, f), 'utf8');
  const t0 = Date.now();
  const r = await compiler.compile(code, { name: id });
  const ms = Date.now() - t0;
  const logTxt = (r.log || []).join('\n');
  fs.writeFileSync(path.join(OUT, id + '.build.log'), logTxt, 'utf8');
  if (r.ok) {
    fs.writeFileSync(path.join(OUT, id + '.hex'), r.hex, 'utf8');
    ok++;
    rows.push(`  ✓ ${id.padEnd(16)} ${String(r.size).padStart(5)} B   ${ms} ms`);
  } else {
    bad++;
    rows.push(`  ✗ ${id.padEnd(16)} 失败: ${(r.errors?.[0]?.msg || '未知').slice(0, 110)}`);
```

```
    try { fs.unlinkSync(path.join(OUT, id + '.hex')); } catch { }
  }
}
console.log('\n' + rows.join('\n'));
console.log(`\n结果: ${ok} 成功 / ${bad} 失败  →  ${OUT}`);
process.exit(bad ? 1 : 0);
/* ==================== tools/reset_password.mjs ==================== */
/**
 * reset_password.mjs — 重置任意账号的密码（运维兜底工具）
 * ------------------------------------------------------------------
 * 场景：教师忘记密码、或学生把初始密码改乱后联系不上教师。
 * 平台没有"邮件找回"（内网部署无邮件服务），因此提供一个本地命令。
 *
 * 用法：
 *   node tools/reset_password.mjs teacher 新密码        # 重置教师密码
 *   node tools/reset_password.mjs 2026001 123456        # 重置学生密码
 *   node tools/reset_password.mjs --list                # 列出所有账号
 *
 * 说明：
 *   · 必须能访问服务器的 data/mculab.db（在服务器本机执行）；
 *   · 重置后默认不强制改密（教师自己设的密码不需要再改一遍），
 *     如需强制，加 --must-change。
 *   · 服务端可以不停机（SQLite 支持多连接写）。
 */
import path from 'node:path';
import process from 'node:process';
import { DB } from '../server/db.js';
const ROOT = path.resolve(import.meta.dirname, '..');
const dbFile = process.env.MCULAB_DB || path.join(ROOT, 'data', 'mculab.db');
const args = process.argv.slice(2).filter(a => a !== '');
const mustChange = args.includes('--must-change');
const rest = args.filter(a => !a.startsWith('--'));
if (args.includes('--list') || rest.length === 0) {
  const db = new DB(dbFile);
  const rows = db.sql.prepare(
    "SELECT username, role, name, class_name, student_no, active FROM users ORDER BY role DESC, class_name,
    student_no"
  ).all();
  console.log(`\n数据库: ${dbFile}`);
  console.log(`共 ${rows.length} 个账号\n`);
  console.log('  账号'.padEnd(20) + '角色'.padEnd(10) + '姓名'.padEnd(12) + '班级');
  console.log('  ' + '-'.repeat(60));
  for (const r of rows) {
    console.log('  ' + String(r.username).padEnd(18) + String(r.role).padEnd(10)
      + String(r.name || '').padEnd(12) + String(r.class_name || ''));
  }
  console.log('\n重置密码: node tools/reset_password.mjs <账号> <新密码>\n');
  db.close();
  process.exit(0);
}
```

```
const [username, newPwd] = rest;
if (!username || !newPwd) {
  console.error('用法: node tools/reset_password.mjs <账号> <新密码> [--must-change]');
  process.exit(1);
}
if (String(newPwd).length < 6) {
  console.error('密码至少 6 位。');
  process.exit(1);
}
const db = new DB(dbFile);
const u = db.getUserByName(username);
if (!u) {
  console.error(`未找到账号「${username}」。用 --list 查看全部账号。`);
  db.close();
  process.exit(1);
}
db.setPassword(u.id, String(newPwd), mustChange ? 1 : 0);
console.log(`✓ 已重置 ${u.role === 'teacher' ? '教师' : '学生'}账号「${username}」的密码`
  + (mustChange ? '（下次登录须修改）' : '') + '。');
db.close();
/* ==================== tools/verify_deploy.mjs ==================== */
/**
 * verify_deploy.mjs — 云端部署核查
 * ================================================================
 * 用途：发布后一条命令确认"线上到底是不是最新版本、能不能用"。
 * 不依赖任何第三方库，只用 Node 内置 fetch。
 *
 * 用法:
 *   node tools/verify_deploy.mjs <分享链接> <教师密码>
 * 例:
 *   node tools/verify_deploy.mjs "https://xxx.app.workbuddy.host/?sharecode=ABC" Mcu5532Lab417
 *
 * 检查项:
 *   1. 站点可达（HTTP 200 且返回平台首页）
 *   2. 分享码会话有效（无分享码应被网关拒绝）
 *   3. 教师登录
 *   4. 新版本必须存在的能力（缺任何一项即判定为"旧版本"）
 *   5. 编译器可用性（云端应内置 Linux 版 SDCC）
 */
const [, , linkArg, pwdArg] = process.argv;
if (!linkArg) {
  console.log('用法: node tools/verify_deploy.mjs "<分享链接>" [教师密码]');
  process.exit(2);
}
const url = new URL(linkArg);
const share = url.searchParams.get('sharecode') || '';
const base = url.origin;
const password = pwdArg || process.env.MCULAB_TEACHER_PWD || '';
/*
 * Cookie 罐：网关的分享码 cookie 前缀不固定（不是 mculab_ 开头），
```

```
 * 必须把响应里的**所有** Set-Cookie 都收下来，否则后续请求会全部 403。
 */
const jar = new Map();
const cookieHeader = () => [...jar.entries()].map(([k, v]) => `${k}=${v}`).join('; ');
function absorb(r) {
  const list = r.headers.getSetCookie ? r.headers.getSetCookie() : [];
  for (const c of list) {
    const nv = c.split(';')[0];
    const i = nv.indexOf('=');
    if (i > 0) jar.set(nv.slice(0, i).trim(), nv.slice(i + 1).trim());
  }
}
const rows = [];
const add = (name, ok, detail = '') => rows.push({ name, ok, detail });
async function req(path, opts = {}) {
  const h = { ...(opts.headers || {}) };
  const ck = cookieHeader();
  if (ck) h.Cookie = ck;
  const r = await fetch(base + path, { ...opts, headers: h, redirect: 'follow' });
  absorb(r);
  return r;
}
async function json(path, opts) {
  const r = await req(path, opts);
  let d = null;
  try { d = await r.json(); } catch { }
  return { s: r.status, d };
}
/* 1. 拿分享码会话 */
let ok = false, body = '';
try {
  const r = await fetch(`${base}/${share ? '?sharecode=' + encodeURIComponent(share) : ''}`, {
    headers: { 'Accept': 'text/html' }, redirect: 'follow'
  });
  absorb(r);
  body = await r.text();
  ok = r.status === 200 && /<title>[^<]*教学仿真实验平台/.test(body);
  add('站点可达并返回平台首页', ok, `HTTP ${r.status}`);
} catch (e) {
  add('站点可达并返回平台首页', false, e.message);
}
/* 2. 无分享码应被拒绝 */
if (share) {
  try {
    const r = await fetch(base + '/');
    const t = await r.text();
    const refused = r.status >= 400 || /链接不完整|链接已失效/.test(t);
    add('无分享码时被网关拒绝（预期行为）', refused, `HTTP ${r.status}`);
  } catch (e) { add('分享码校验', false, e.message); }
}
```

```
/* 3. 登录 */
if (password) {
  const r = await json('/api/auth/login', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: 'teacher', password })
  });
  add('教师账号可登录', r.s === 200 && !!r.d?.user, r.s === 200 ? `角色 ${r.d.user.role}` : (r.d?.error || `HTTP ${r.
    s}`));
} else {
  add('教师账号可登录', null, '未提供密码，跳过');
}
/* 4. 版本能力探测：新功能接口必须存在 */
const features = [
  ['实验任务', '/api/teacher/assignments'],
  ['成绩册', '/api/gradebook'],
  ['开放性命题', '/api/open-problems'],
  ['AI 知识库', '/api/kb/docs'],
  ['联网搜索配置', '/api/websearch/config'],
  ['高频错误统计', '/api/teacher/errors']
];
let newCount = 0, blocked = 0;
for (const [label, path] of features) {
  const r = await json(path);
  // 404 = 路由不存在（旧版本）；403/401 = 被网关或权限挡住，无法判断
  if (r.s === 404) add(`能力：${label}`, false, '接口不存在（旧版本）');
  else if (r.s === 403 || r.s === 401) { blocked++; add(`能力：${label}`, null, `HTTP ${r.s}（会话无效，无法判断）`); }
  else { newCount++; add(`能力：${label}`, true, `HTTP ${r.s}`); }
}
add('判定为新版本', blocked ? null : (newCount === features.length),
  blocked ? `${blocked} 项因会话无效无法判断` : `${newCount}/${features.length} 项能力就绪`);
/* 5. 编译器 */
const st = await json('/api/compiler/status');
if (st.s === 403 || st.s === 401) add('云端 C 编译器可用', null, `HTTP ${st.s}（会话无效）`);
else add('云端 C 编译器可用', st.d?.sdccUsable === true,
  st.d ? (st.d.sdccUsable ? String(st.d.version || '').slice(0, 40) : (st.d.error || '不可用')) : `HTTP ${st.s}
    `);
/* 输出 */
const pad = (s, n) => String(s).padEnd(n, ' ');
console.log('\n===== 云端部署核查 =====');
console.log('地址 :', base);
console.log('分享码:', share || '(无)');
console.log('');
for (const r of rows) {
  const mark = r.ok === null ? '-' : (r.ok ? '✓' : '✗');
  console.log(`  ${mark} ${pad(r.name, 34)} ${r.detail}`);
}
const failed = rows.filter(r => r.ok === false);
console.log('');
console.log(failed.length ? `结果：${failed.length} 项未通过` : '结果：全部通过');
process.exit(failed.length ? 1 : 0);
```